

VUSB

Universal Serial Bus (USB) Synthesizable HDL Core Reference Manual

Revision 9911b

Disclaimer

VAutomation Reserves the right to make changes at any time, without notice, to improve design or performance and supply the best product possible. VAutomation Inc. makes no warrant for the use of its products and assumes no responsibility for any errors which may appear in this document nor does it make a commitment to update the information contained herein. VAutomation products are not authorized for use in life support devices or systems unless a specific written agreement pertaining to such use is executed between the manufacturer and an officer of VAutomation. Nothing contained herein shall be construed as a recommendation to use any product in violation of existing patents, copyrights or other rights of third parties. No license is granted by implication or otherwise under any patent, patent rights or other rights, of VAutomation Inc. All trademarks are trademarks of their respective companies.

Copyright © 1996-2000 VAutomation Inc. Nashua NH. All rights reserved.

VAutomation Inc.
402 Amherst St
Suite 100
Nashua NH 03063
(603) 882-2282
FAX: 882-1587
<http://www.VAutomation.com>

Table of Contents

1 Installation	1
2 Release Notes	1
3 Introduction	4
4 Features	4
5 Getting Started	5
6.1 VUSB Description	5
6.2 USB_SIE	6
6.3 Micro-Processor Interface	6
6.4 DPLL	6
6.5 V8-uRISC Core	7
7 VUSB Hardware/Software Interface	7
7.1 Buffer Descriptor Table	7
7.2 Addressing Buffer Descriptor Table Entries	8
7.3 Buffer Descriptor Formats	8
7.4 USB Transaction	10
8 VUSB Control Register Block	12
8.1 Register Bit Definitions	13
8.2 Interrupt Status Register (INT_STAT 00)	13
8.3 Interrupt Enable Register (INT_ENB 01)	14
8.4 Error Interrupt Status Register (ERR_STAT 02)	15
8.5 Error Interrupt Enable Register (ERR_ENB 03)	17
8.6 Status Register (STAT 04)	17
8.7 Control Register (CTL 05)	18
8.8 Address Register (ADDR 06)	19
8.9 BDT Page Register (BDT_PAGE 07)	20
8.10 Frame Number Register (FRM_NUML 08, FRM_NUMH 09)	20
8.11 Token Register (TOKEN 0A)	20
8.12 SOF Threshold Register (SOF_THLD 0B)	21
8.13 Endpoint Control Register (EP+CTLn 10-1F)	22
9 Host Mode Operation	24
9.1 USB Pull-up/pull-down resistors in Host Mode	24
9.2 Sample Host Mode operation	25
10 Signal Descriptions	26
10.1 Generic Microprocessor Bus Interface Signals	27
10.2 USB Interface Signals	29
11 Bus Cycles	30
11.1 VUSB Slave Write Bus Cycle	30

11.2 VUSB Slave Read Bus Cycle	30
11.3 VUSB Master Write Bus Cycle	31
11.4 VUSB Master Read Bus Cycle	31
12 Timing	33
13 Synthesizing the VUSB	33
13.1 Makefile	34
13.2 VUSB File Hierarchy	34
13.3 VUSB Configurations	36
13.4 VUSB Programmable Control	37
14 Test Bench Description	40
14.1 VUSB Test Bench File Hierarchy and Architectures	40
14.2 Programmable Control of the Test Bench	42
14.3 Behavioral Host Controller API	44
14.3.1 USB_RESET	44
14.3.2 USB_RESUME	44
14.3.4 BUILD_HC_PACKET	45
14.3.5 DATA_IN_PACKET	46
14.3.6 DATA_OUT_PACKET	47
14.3.7 HANDSHAKE_IN_PACKET	47
14.3.8 HANDSHAKE_OUT_PACKET	48
14.3.9 GET_DEVICE_DESC	48
14.3.10 GET_HUB_DESC	49
14.3.11 GET_CONFIG_DESC	49
14.3.12 GET_DEVICE_STATUS	50
14.3.13 GET_ENDPOINT_STATUS	50
14.3.14 GET_VALUE	51
14.3.15 SET_ADDRESS	52
14.3.16 SET_VALUE	52
14.3.17 SET_FEATURE	53
14.3.18 CLEAR_FEATURE	54
14.3.19 GET_DATA	55
14.3.20 SET_DIAG_DESC	56
14.3.21 GET_DIAG_DESC	58
14.4 Executing the Test Bench in VHDL	58
14.5 Executing the Test Bench in Verilog	60
14.6 V8 C-Assembly language software	61
15 Tips and Techniques	62
15.1 Debugging VUSB software in an HDL simulator	62
15.2 Interfacing to Off-Chip SRAM	63
16 IntelliCore™ Prototype System	63

1 Installation

Typically the VUSB core is shipped on a single DOS compatible 3.5 inch floppy disk. The following description details how to install the VUSB core onto a PC. If you have requested shipment on other media, follow the instructions on the label of the media shipped. You will need the Authorization Key from your signed license agreement to install the VUSB core.

If you have any problems or question installing the VUSB core, please call our hotline at (603) 882-2282 or via Internet Email at support@VAutomation.com.

To install VAutomation products, run Setup.exe on the floppy disk.

Setup will ask you where to put the VAutomation files.

Setup will recommend C:\vhdfiles but you may pick another directory and/or another drive. You may even specify a network drive such as \\Server\user_name\vhdfiles.

Once you have specified where the files are to be installed, click "Next".

You may then view our license agreement, and signify your acceptance of it by clicking "Agree".

Then enter your Authorization Key and click "Next".

Then enter your name and click "Next".

Then enter your company name and click "Next".

Then enter your site name and click "Finish".

All VAutomation files will be copied to the specified directory. There are no hidden files. No files are copied to any other directory.

If you wish to remove all the VAutomation files, run uninstall.exe.

A Unix "make" utility file is shipped with the VUSB. This utility provides command line examples on how to build a simulation database and synthesize the VUSB. Depending on your CAE tool set, you may need to setup various environment variables and your \$PATH to properly use the Makefile commands. See the comments in the Makefile file for more information.

Type "make sim" to create a simulation database.

Type "Make syn" to synthesize the VUSB model.

2 Release Notes

January 2000 Rev 9911b Release

Bug fix for BDT write back PID failure in the presents of excessive (5 us) arbitration delay. This problem was observed in systems employing a bridge to a remote memory system that held off the BDT write operation until after a subsequent tokens have been received on the USB bus.

January 2000 Rev 9911a Release

Changed host_wo_hub generic to a software control endpoint zero control register bit. This change allows the host to detect directly connected low speed devices and respond accordingly. This change effects host mode only.

November 1999 Rev 9911 Release

Improved host_ctl code to allow the full checklist_enabled simulations to be run.

Improved the DPLL Jitter rejection to exceed the USB Jitter specification. It is recommended

that the new design be adopted as the original DPLL design did not allow jitter margin for the analog receiver circuits. The following table shows the simulated performance of the old and new DPLL designs. Note: The USB specification for Jitter is +/-9ns with a frequency tolerance of +/-209 ps per bit for the paired transitions that are used to adjust the DPLL. No time budget has been made for the analog receiver circuits in this analysis.

-- Total JITTER	Frequency Tolerance	Frequency Tolerance
--	Unimproved Design	Jitter Hardened design
-- +/ - 0 ns	+/- 1.4 ns	+/- 1.4 ns
-- +/ - 5 ns	+/- 800 ps	+/- 1.4 ns
-- +/ - 9 ns	+/- 210 ps	+/- 800 ps
-- +/ - 10 ns	+/- 0 ps	+/- 700 ps
-- +/ - 11 ns	can not track	+/- 600 ps
-- +/ - 12 ns	can not track	+/- 400 ps
-- +/ - 13 ns	can not track	+/- 200 ps
-- +/ - 14.75 ns	can not track	+/- 100 ps
--		

Removed the the empty USB-HUB from the top level asic_v8 in non-hub products.
Fixed a bug in the SIE that resulted in bit stuff error (bst_err) interrupts being generated if there was noise on the bus while the SIE was waiting for a token sync field.

September 1999 Rev 9909 Release

Added code to support host mode operation through a hub to high or low speed devices.
Change host interface endpoint 0 register definition to support host operation to interrupt endpoints.
Changed the operation of the returned TOK_PID field of the BDT for host operations to allow return of the transfer status.
Renamed the simulation test bench configurations. Supported configurations are now:

- a_usb_hsdev_async - high speed device with asynchronous V8 interface
- a_usb_hsdev_sync - high speed device with synchronous V8 interface
- a_usb_hc_hsdev_async - host controller with high speed device and asynchronous V8 interface.
- a_usb_hc_hsdev_sync - host controller with high speed device and synchronous V8 interface.
- a_usb_hc_lsdev_async - host controller with low speed device and asynchronous V8 interface.
- a_usb_hc_lsdev_sync - host controller with low speed device and synchronous V8 interface.

July 1999 Rev 9907 Release

Added code to allow the VUSB Host to support directly connected low speed devices.
Fixed the operation of the CRC5, CRC16 and DFN8 error interrupt signals.
Cut the turn around time from packet receipt to response by one cycle to allow more time for gate level delays.

December 8, 1998 Rev 9812 Release

VUSB update to support VUSB_HUB release.
Updated the DPLL to reject SE0 pulses of up to 20ns on DPLUS and DMINUS (dpllnrzi).
Fixed constant width for zero check on Host mode SOF counter (usb_sie).

November 20, 1998: Rev 9811 Release

Updated the DPLL to increase meta-stability hardening.

Implemented the USB Rev 1.1 Specification Protocol and endpoint halt STALL functions.
This caused us to add a BDT-STALL bit to the BDT, and convert the Host token busy control bit to a dual use bit TXD-Suspend for target applications and Token-Busy for Host applications.
Added hardware support for USB Suspend detection, and Remote wake up signaling.

July 22, 1998: Rev 9807 Release

June 3, 1998: Rev 9805 Release

May 26, 1998 Rev 9804 Release

April 29, 1998 Rev 1.0
Initial Release. Implements USB Rev 1.0 including HOST_MODE.

3 Introduction

The VUSB IntelliCore™ is a complete solution for implementing a USB peripheral application in any ASIC or FPGA technology. Verified in silicon and fully compliant with the USB 1.0 specification, the VUSB IntelliCore™ will reduce your development cost and risk while speeding your product to market. Why commit valuable engineering resources to interpreting and verifying the USB specification when VAutomation has already done this for you?

As with all IntelliCores™, the VUSB is an Intelligent Core. Much of the USB protocol is implemented in software which executes on an embedded 8-bit RISC microprocessor, the V8-uRISC. The use of a microprocessor-based architecture has several advantages over hardware-only solutions, including:

- ! Supports any number of configurations, interfaces and alternate settings with no increase in gate count
- ! Relieves your application software and processor from USB housekeeping tasks
- ! Sample USB Software Drivers included
- ! Easily supports any USB Device Class specification

The V8-uRISC processor has ample performance to execute your application and includes a complete package of microcontroller peripherals to create a One-Chip Solution. Software Development is fully supported with an Assembler, C Compiler, JTAG debugger/emulator and an FPGA Prototype Development System.

Written in technology independent, fully synchronous VHDL or Verilog, the VUSB to be easily customized for specific customer needs. Synthesis is supported by virtually any HDL logic synthesizer such as the Synopsys Design Compiler. Technology Independence allows the VUSB to be targeted to virtually any ASIC technology supported by the logic synthesizer. Technologies from FPGAs to +5V CMOS or 3.3V submicron CMOS are all possible target technologies for the VUSB.

4 Features

- ! Fully USB 1.1 compliant Device Core
- ! 16 Bidirectional End Points
- ! 8-Bit Embedded RISC Processor
- ! DMA or FIFO data stream interfaces
- ! USB driver software source code (6.5KByte Typical)
 - ! USB Audio Class driver software source code
 - ! USB Printer Class driver software source code
 - ! USB Chapter 9 driver software source code
- ! Optional Host-Mode logic for emulating a PC host
- ! USB logic requires approximately 5,000 gates
- ! V8 RISC processor requires approximately 3,000 gates
- ! Technology and ASIC Vendor Independent
- ! Fully Synchronous and Static Design
- ! No Internal Tri-State Busses
- ! Low Power Consumption
- ! Push-Button Synthesis
- ! Push-Button Timing Analysis
- ! Complete Test Bench Supplied
- ! Supports Embedded Host Controller
- ! Intellicore™ Prototype System available

5 Getting Started

There are two different ways to begin integrating the VUSB core into your ASIC or FPGA design. The first scenario typically begins by instantiating the VUSB_TOP.V(HD) (or VUSB_TOP.V for Verilog users) into your ASIC and adding other peripherals as needed. The simulation environment and test bench will be of your own design. To save time, we recommend that you look through the test bench, assembly code, peripherals and simulation models and reuse these items where appropriate. This approach is best suited to applications which are significantly different than those provided in the sample simulation environment provided.

The second scenario for integrating the VUSB into your ASIC is to use VAutomation's test bench and add your logic within the provided simulation environment. In this case you will want to start with the ASIC_IO.V(HD) module and redefine this module to match the IOs of your ASIC. Then add your own logic to the ASIC_V8.V(HD) file and add, remove or customize the other peripherals provided. This method will save you time since a fairly complete simulation environment has already been provided by VAutomation.

Whichever method you choose, the main point here is that you have purchased much more than just a synthesizable VUSB core. Numerous synthesizable peripherals and behavioral simulation models have been provided as well. A complete board level simulation environment which can easily be expanded to multiple board or system simulations has been provided. The environment provided is a fairly complete simulation of the IntelliCore Prototype System which is available from VAutomation for full emulation of your ASIC in FPGAs.

6 Functional Description

6.1 VUSB Description

The VUSB core is comprised of four major functional blocks.

USB_SIE	USB Serial Interface Engine
Micro-Processor Interface	Generic microprocessor interface with DMA
DPLL	Digital Phase-Locked Loop logic
V8-uRISC Core	8-bit Embedded microprocessor

Each of these functional block are described in this section. A block diagram of the VUSB core is shown in Figure 1.

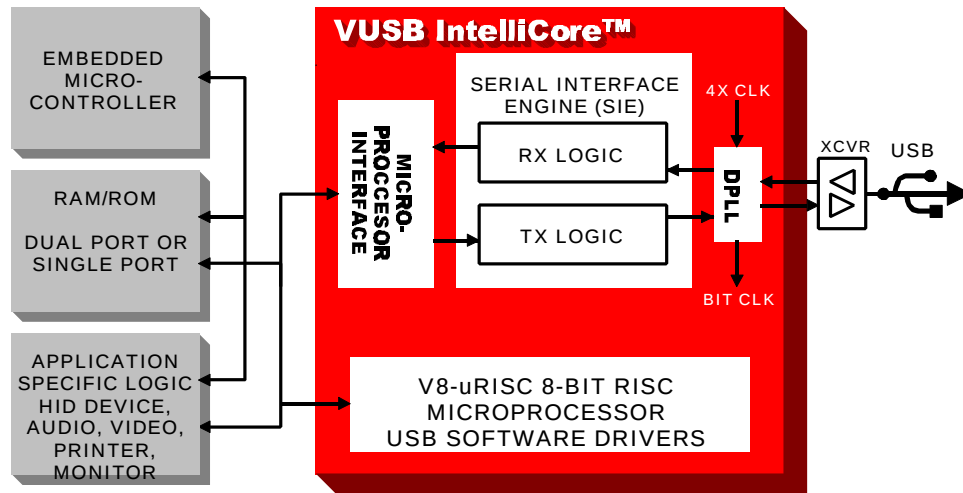


Figure 1

6.2 USB_SIE

This USB_SIE is comprised of two major sections of synthesizable HDL code, the Tx Logic and Rx Logic. The Tx Logic formats and transmits data packets that are built in memory by the V8 RISC core. These packets are converted from a parallel to serial data stream. All of the necessary USB data formatting is performed by the Tx Logic. This includes, NRZI encoding, bit-stuff, CRC computation, addition of SYNC field and EOP.

The Rx Logic receives USB data and stores these packets in memory for processing by the V8 RISC core. Serial USB data is converted to a byte wide parallel data stream and stored in system memory. The receive logic decodes an NRZ USB serial data stream, performs bit-stuff removal, CRC check, PID check, and other USB protocol layer checks.

6.3 Micro-Processor Interface

The Micro-Processor Interface within the VUSB is made up of a slave and master interface. The slave interface consists of a number of control and configuration registers for the VUSB. VUSB internal registers can be accessed via a simple micro-processor interface, see Section 8 for more information. The master interface is the integrated DMA controller which is used transfer the packet data to and from memory. The DMA controller allows USB endpoint data to be transferred in a very efficient manner, while limiting processor involvement.

6.4 DPLL

The Digital Phase Lock Loop is responsible for maintaining a 12-MHz clock source that is locked to the USB data stream. The DPLL requires a 48 MHz clock to 4X oversample the USB data stream to detect transitions. These transitions are used to synthesize a nominally 12-MHz USB clock. The DPLL also detects single ended zeros, end of packet strobes, and NRZI decode of the serial data stream for the Rx Logic. All of the outputs from the DPLL are synchronized to the 12-MHz clock which allows for a seamless connection to the VUSB core.

6.5 V8-uRISC Core

The brains of the VUSB Intellicore™ is provided by the V8-uRISC microprocessor. The V8 core provides the intelligence necessary to implement the complex USB protocol. The V8 can be implemented in less than 3000 gates and can significantly offload your application processor from the time-critical and interrupt intensive demands of USB. Applications with modest CPU requirements can use the V8 as their application processor for the lowest cost implementation.

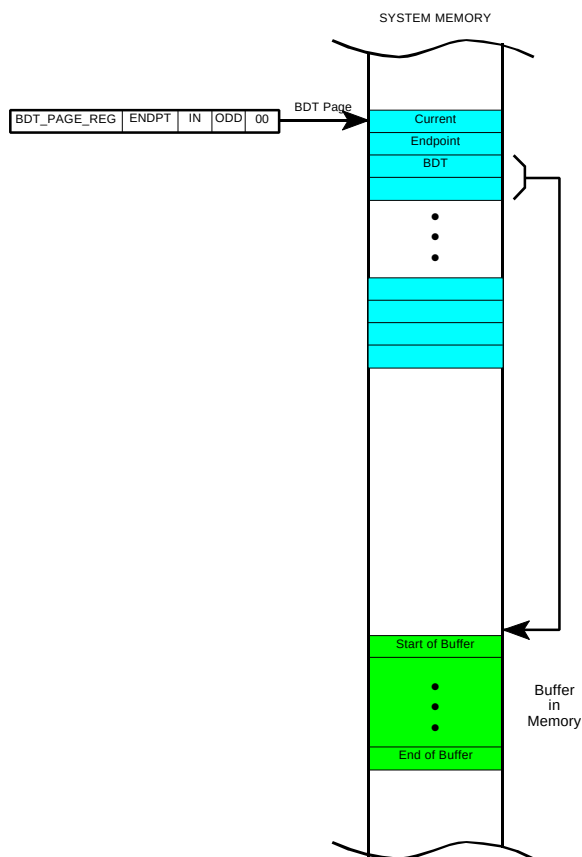
Because of the V8's intelligence, virtual FIFOs can be built in memory using VAutomation's Buffer Descriptor Architecture to implement USB data packets from 0 -1024 bytes. Software drivers are also provided that allow the VUSB users to rapidly implement USB peripherals. These drivers include USB Audio Class, USB Printer Class, and USB Chapter 9.

7 VUSB Hardware/Software Interface

The VUSB core uses a combination of hardware and software to efficiently implement USB target applications. While the USB Serial Interface Engine handles the low level USB Protocol Layer, the V8 can handle the higher level USB Device Framework, buffer management, and peripheral dependent functions.

This Hardware/Software interface of the VUSB has both a slave and master interface. The slave interface is comprised of the Control Registers Block (CRB) used to configure the VUSB and to provide status and interrupts to the V8. The master interface is through the VUSB integrated DMA controller

which interrogates the Buffer Descriptor Table (BDT), and transfers USB data to or from system memory. The Buffer Descriptor Table allows the V8 and VUSB to efficiently manage multiple endpoints with very little processor overhead.



7.1 Buffer Descriptor Table

To efficiently manage USB endpoint communications the VUSB implements a Buffer Descriptor Table (BDT) in system memory. The BDT resides on a 256 byte boundary in system memory and is pointed to by the BDT Page Register. Every endpoint direction requires two four byte Buffer Descriptor entries. Therefore, a system with 16 fully bi-directional endpoints would require 256 bytes of system memory to implement the BDT. The two Buffer Descriptor (BD) entries allows for an EVEN BD and ODD BD entry for each endpoint direction. This allows the V8 to process one BD while the VUSB is processing the other BD. Double buffering BDs in this way allows the VUSB to easily transfer data at the maximum throughput provided by USB. The V8 intelligently manages buffers for the VUSB by updating the BDT when needed. This allows the VUSB to efficiently handle data transmission (IN TOKENS) and reception

Figure 2

(OUT/SETUP TOKENS). While the V8 can perform communication overhead processing and other function dependent applications. Because the buffers are shared between the V8 and the VUSB a simple semaphore mechanism is used to distinguish who is allowed to update the BDT and buffers in system memory. A semaphore bit, the OWN bit, is cleared to 0 when the BD entry is “owned” by the V8. The V8 is allowed read and write access to the BD entry and the buffer in system memory when the OWN bit is 0. When the OWN bit is set to 1, the BD entry and the buffer in system memory are owned by the VUSB. The VUSB now has full read and write access and the V8 should not modify the BD or its corresponding data buffer. The BD also contains indirect address pointers to where the actual buffer resides in system memory. This indirect address mechanism is shown in Figure 2.

7.2 Addressing Buffer Descriptor Table Entries

To access endpoint data via the VUSB or V8, the addressing mechanism of the Buffer Descriptor Table must be understood. As stated earlier, the Buffer Descriptor Table occupies up to 256 bytes of system memory. Sixteen bidirectional endpoints can be supported with a full BDT of 256 bytes. Eight bytes are needed for each USB endpoint direction. Applications with less than 16 End Points will require less RAM to implement the BDT. The BDT Page Register points to the starting location of the BDT. The BDT must be located on a 256 byte boundary in system memory. All enabled IN and OUT endpoint BD entries are indexed into the BDT to allow easy access via the VUSB or V8.

When the VUSB receives a USB token on an enabled endpoint it uses its integrated DMA controller to interrogate the BDT. The VUSB must read the corresponding endpoint BD entry and determine if it owns the BD and corresponding buffer in system memory. To compute the entry point in to the BDT, the BDT_PAGE register is concatenated with the current endpoint and the IN and ODD fields to form a 16-bit address. This address mechanism is shown in **Table 1**.

Table 1 BDT Address Generation

15:8	7:4	3	2	1:0
BDT_PAGE REGISTER	END POINT	IN	ODD	0 0

BDT_PAGE BDT_PAGE register in the Control Register Block
END_POINT END POINT field from the USB TOKEN
IN 1 for an IN TOKEN, 0 for an OUT or SETUP TOKEN
ODD This bit is maintained within the VUSB SIE. It corresponds to the buffer that is currently in use. The buffers are used in a ping-pong fashion.

7.3 Buffer Descriptor Formats

The Buffer Descriptors provide endpoint buffer control information for the VUSB and V8. The Buffer Descriptors have different meaning based on who is reading the BD in memory. The VUSB Controller uses the data stored in the BDs to determine:

- Who owns the buffer in system memory
- Data0 or Data1 PID
- Release Own upon packet completion
- No address increment (FIFO Mode)
- Data toggle synchronization enable
- How much data is to be transmitted or received
- Where the buffer resides in system memory

While the Microprocessor uses the data stored in the BDs to determine:

- Who owns the buffer in system memory
- Data0 or Data1 PID
- The received TOKEN PID
- How much data was transmitted or received
- Where the buffer resides in system memory

The format for the BD is shown in Table 2.

Table 2 VUSB Buffer Descriptor Format

7	6	5	4	3	2	1	0
OWN	DATA0/1	KEEP/ TOK_PID[3]	NINC/ TOK_PID[2]	DTS/ TOK_PID[1]	BDT_STALL TOK_PID[0]	BCH	
BCL							
ADDRL							
ADDRH							

Field	Description
OWN	If OWN=1 VUSB has exclusive access to the BD. If OWN=0 the V8 has exclusive access to the BD. This OWN bit determines who currently owns the buffer. The SIE generally writes a 0 to this bit when it has completed a token, except when KEEP=1. The VUSB ignores all other fields in the BD when OWN=0. The V8 has access to the entire BD when OWN=0. This byte of the BD should always be the last byte the V8 updates when it initializes a BD. Once the BD has been assigned to the VUSB, the microprocessor should not change it in any way.
DATA0/1	This bit defines if a DATA0 field (DATA0/1=0) or a DATA1 (DATA0/1=1) field was transmitted or received. It is unchanged by the VUSB.
KEEP/ TOK_PID[3]	If KEEP equals 1, once the OWN bit is set it will remain owned by the VUSB forever. KEEP must equal 0 to allow the VUSB to release the BD when a token has been processed. Typically this bit is set to 1 with ISO endpoints that are feeding a FIFO. The V8 is not informed that a token has been processed, the data is simply transferred to or from the FIFO. The NINC bit is normally also set when KEEP=1 to prevent address increment. If KEEP=1 this bit is unchanged by the VUSB, otherwise bit 3 of the current token PID is written back in to the BD by the VUSB.
NINC/ TOK_PID[2]	The No INCRement bit disables the DMA engine address increment. This will force the DMA engine to read or write from the same address. This is useful for endpoints when data needs to be read from or written to a single location such as a FIFO. Typically this bit is set with the KEEP bit for ISO endpoints that are interfacing to a FIFO. If KEEP=1 this bit is unchanged by the VUSB, otherwise bit 2 of the current token PID is written back in to the BD by the VUSB.
DTS/ TOK_PID[1]	Setting this bit will enable the VUSB to perform Data Toggle Synchronization. When this bit is 0 no Data Toggle Synchronization is performed. If KEEP=1 this bit is unchanged by the VUSB, otherwise bit 1 of the current token PID is written back in to the BD by the VUSB.

BDT-STALL TOK_PID[0]	Setting this bit will cause the VUSB to issue a STALL handshake if a token is received by the SIE that would use the BDT in this location. The BDT is not consumed by the SIE (the owns bit remains and the rest of the BDT are unchanged) when a BDT-STALL bit is set. If KEEP=1 this bit is unchanged by the VUSB, otherwise bit 0 of the current token PID is written back in to the BD by the VUSB
TOK_PID	The current token PID is written back in to the BD by the VUSB when a transfer completes. The Values values written back are the token PID values from the USB specification: 0x1 for an Out token, 0x9 for and In token or 0xd for a Setup token. In host mode this field is used to report the last returned PID or a transfer status indication. The possible values returned are : 0x3 Data0, 0xb Data1, 0x2 Ack, 0xe Stall, 0xa Nak, 0x0 Bus Timeout, 0xf Data Error.
BCH[9:8]	The Byte Count High bits represent the high order bits of the 10-bit Byte Count. The VUSB SIE will change this field upon the completion of an OUT or SETUP token with the byte count of the data received.
BCL[7:0]	The Byte Count Low bits represent the low order byte of the 10-bit Byte Count. The BCH and BCL together form the 10-bit byte count. This represents the number of bytes that will be transmitted for an IN TOKEN or received during an OUT TOKEN. Valid byte counts are 0 - 1023. The VUSB SIE will change this field upon the completion of an OUT or SETUP token with the actual byte count of the data received.
ADDRL[7:0]	The Address Low bits represent the low order byte of the 16-bit buffer address in system memory. These bits are unchanged by the VUSB.
ADDRH [15:8]	The Address High bits represent the high order byte of the 16-bit buffer address in system memory. The ADDRH and ADDRL together form the 16-bit address where the buffer resides in system memory. This is the address that the VUSB DMA engine will read or write data. These bits are unchanged by the VUSB.

7.4 USB Transaction

When the VUSB transmits or receives data it will first compute the BDT address using the address generation shown in Table 2. Once the BDT has been read and if the OWN bit equals 1, the SIE will DMA the data to or from the buffer pointed to by the ADDR field of the BD. When the TOKEN is complete, the VUSB will update the BDT and change the OWN bit to 0 if KEEP is 0. The STAT register is updated and the TOK_DNE interrupt is set. When the V8 processes the TOK_DNE interrupt it reads the status register, this gives the V8 all the information it needs to process the endpoint. At this point the V8 will allocate a new BD so additional USB data can be transmitted or received for that endpoint, and process then the last BD. Figure 3 shows a time line how a typical USB token would be processed.

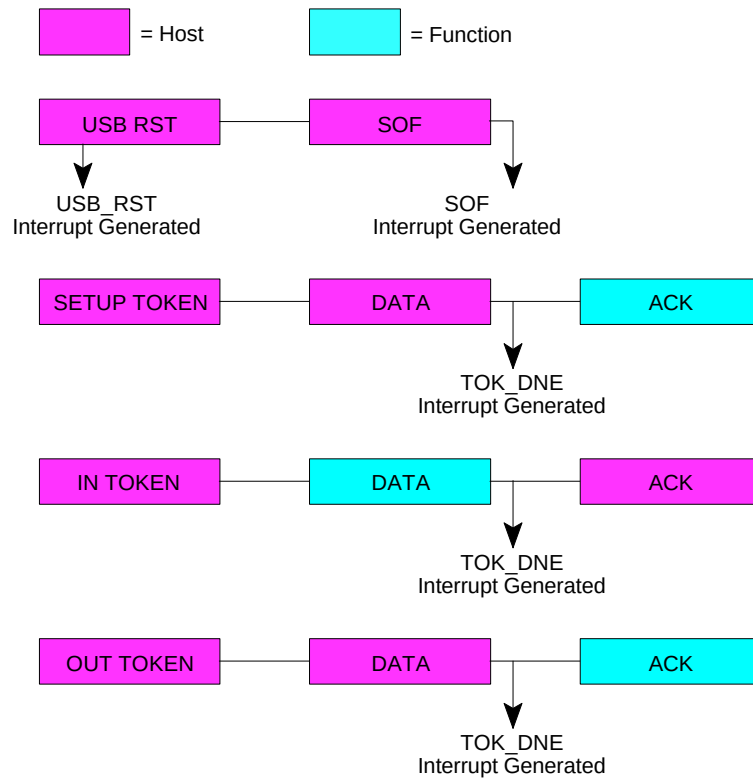


Figure 3

8 VUSB Control Register Block

This section documents the VUSB control registers. **Table 3** shows the memory map of the Control Register Block (CRB). Registers dedicated to HOST MODE are shown in grey. All registers are read/write unless noted.

Table 3 CRB Memory Map

Register	Offset
INT_STAT	00\h
INT_ENB	01\h
ERR_STAT	02\h
ERR_ENB	03\h
STAT	04\h
CTL	05\h
ADDR	06\h
BDT_PAGE	07\h
FRM_NUML	08\h
FRM_NUMH	09\h
TOKEN	0A\h
SOF_THLDL	0B\h
SOF_THLDH	0C\h
ENDPT0	10\h
ENDPT1	11\h
ENDPT2	12\h
ENDPT3	13\h
ENDPT4	14\h
ENDPT5	15\h
ENDPT5	16\h
ENDPT7	17\h
ENDPT8	18\h
ENDPT9	19\h
ENDPT10	1A\h
ENDPT11	1B\h
ENDPT12	1C\h
ENDPT13	1D\h
ENDPT14	1E\h
ENDPT15	1F\h

8.1 Register Bit Definitions

The following table provides a quick reference for the CRB register bit definitions.

Table 4 CRB Quick Reference

Register	Offset	Bit							
		7	6	5	4	3	2	1	0
INT_STAT	00h	STALL	ATTACH	RSVD	SLEEP	TOK_DNE	SOF_TOK	ERROR	USB_RST
INT_ENB	01h	STALL	ATTACH	RSVD	SLEEP	TOK_DNE	SOF_TOK	ERROR	USB_RST
ERR_STAT	02h	BTS_ERR	RSVD	DMA_ERR	BTO_ERR	DFN8	CRC16	CRC5/EO F	PID_ERR
ERR_ENB	03h	BTS_ERR	RSVD	DMA_ERR	BTO_ERR	DFN8	CRC16	CRC5/EO F	PID_ERR
STAT	04h	ENDP				IN	ODD	0	0
CTL	05h	JSTATE	SE0	TxdSuspend TokBusy	RESET	HOST_ MODE_EN	RESUME	ODD_RST	USB_EN
ADDR	06h	LS_EN	ADDR6	ADDR5	ADDR4	ADDR3	ADDR2	ADDR1	ADDR0
BDT_PAGE	07h	BDT_BA15	BDT_BA1 4	BDT_BA13	BDT_BA12	BDT_BA11	BDT_BA1 0	BDT_BA9	BDT_BA8
FRM_NUML	08h	FRM7	FRM6	FRM5	FRM4	FRM3	FRM2	FRM1	FRM0
FRM_NUMH	09h	FRM15	FRM14	FRM13	FRM12	FRM11	FRM10	FRM9	FRM8
TOKEN	0Ah	TOKEN_PID				TOKEN_ENDPT			
SOF_THLD	0Bh	CNT7	CNT6	CNT5	CNT4	CNT3	CNT2	CNT1	CNT0
ENDPT_CTL	10h- 1fh	HOST_WO_ HUB	RETRY_DI S		EP_CTL_DI S	EP_EN_OU T	EP_EN_IN	EP_STALL	EP_HSHK

The remainder of section describes the bit definitions for each register contained in the CRB. The format is the same for each register. There is a quick reference in the first four lines that describe the access, address, code reference, and initialization. The access field describes what type of accesses maybe performed to that register. Valid types of access are: read/write, read only, or write only. The address reference defines the address location of the register in the CRB. The code reference states the name of the register in the HDL source code and the file name is contained in parentheses. This allows designers to easily display internal register values during simulation. The initialization field describes the value the register will be initialized to at reset. A brief functional description of the register, detailed bit definitions, and operation are also included.

8.2 Interrupt Status Register (INT_STAT 00)

Access: Read\Write

Address: 00h

Code Reference: int_stat (up_int)

Initialization: 00h

The Interrupt Status Register (INT_STAT) contains bits for each of the interrupt sources within the VUSB. Each of these bits are qualified with their respective interrupt enable bits (see the Interrupt Enable Register). All bits of the register are logically OR'ed together to form a single interrupt source for the microprocessor. Once an interrupt bit has been set it may only be cleared by writing a one to the respective interrupt bit. This register will contain the value of 00h after a reset.

7	6	5	4	3	2	1	0
STALL	ATTACH	RESUME	SLEEP	TOK_DNE	SOF_TOK	ERROR	USB_RST

Bit Name	Description
USB_RST	This bit is set when the VUSB has decoded a valid USB reset. This will inform the microprocessor to write 00h into the address register and enable endpoint 0. USB_RST is set once a USB reset has been detected for 2.5 microseconds. It will not be asserted again until the USB reset condition has been removed, and then reasserted.
ERROR	This bit is set when any of the error conditions with in the ERR_STAT register has occurred. The microprocessor must then read the ERR_STAT register to determine the source of the error.
SOF_TOK	This bit is set if the VUSB has received a Start Of Frame (SOF) token.
TOK_DNE	This bit is set when the current token being processing is complete. The microprocessor should immediately read the STAT register to determine the End Point and BD used for this token. Clearing this bit (by writting a one) causes the STAT register to be cleared or the STAT holding register to be loaded into the STAT register.
SLEEP	This bit is set if the VUSB has detected a constant idle on the USB bus signals for 3 ms. The sleep timer is reset by activity on the USB bus.
RSVD	This bit will remain low. It is reserved for future use.
ATTACH	This bit is set if the VUSB has detected a attach of a USB peripheral. This signal is only valid if HOST_MODE_EN is true. This interrupt is useful to signify that a peripheral is now present and must be configured. The ATTACH interrupt is asserted if there have been no transitions on the USB for 2.5us and the current bus state is not SE0.
STALL	The stall interrupt is used in target and host modes. In target mode it is asserted when a STALL handshake is sent by the SIE. In host mode this bit is set if the VUSB has detected a STALL acknowledge during the handshake phase of a USB transaction. This interrupt is useful to determine if the last USB transaction was completed successfully or stalled.

8.3 Interrupt Enable Register (INT_ENB 01)

Access: Read\Write

Address: 01h

Code Reference: int_enb (up_int)

Initialization: 00h

The Interrupt Enable Register (INT_ENB) contains enable bits for each of the interrupt sources within the VUSB. Setting any of these bits will enable the respective interrupt source in the INT_STAT register. This register will contain the value of 00h after a reset, i.e. all interrupts disabled.

Table 4 INT_ENB Register Format

7	6	5	4	3	2	1	0
STALL	ATTACH	RESUME	SLEEP	TOK_DNE	SOF_TOK	ERROR	USB_RST

Interrupt Mask Register (INT_ENB) Bit Definitions:

USB_RST Setting this bit will enable USB_RST interrupts.
ERROR Setting this bit will enable ERROR interrupts.
SOF_TOK Setting this bit will enable SOF_TOK interrupts.
TOK_DNE Setting this bit will enable TOK_DNE interrupts.
SLEEP Setting this bit will enable SLEEP interrupts.
RESUME Setting this bit will enable RESUME interrupts.
ATTACH Setting this bit will enable ATTACH interrupts.
STALL Setting this bit will enable STALL interrupts.

8.4 Error Interrupt Status Register (ERR_STAT 02)

Access: Read\Write

Address: 02\h

Code Reference: err_stat (up_int)

Initialization: 00\h

The Error Interrupt Status Register (ERR_STAT) contains bits for each of the error sources within the VUSB. Each of these bits are qualified with their respective error enable bits (See Error Enable Register page). The result is OR'ed together and sent to the ERROR bit of the INT_STAT register. Once an interrupt bit has been set it may only be cleared by writing a one to the respective interrupt bit. Each bit is set as soon as the error condition is detected. Thus, the interrupt will typically not correspond with the end of a token being processed. This register will contain the value of 00\h after a reset.

Table 5 ERR_STAT Register Format

7	6	5	4	3	2	1	0
BTS_ERR	RSVD	DMA_ERR	BTO_ERR	DFN8	CRC16	CRC5/EOF	PID_ERR

Error Interrupt Status Register (ERR_STAT) Bit Definitions:

PID_ERR The PID check field failed.

CRC5/EOF This error interrupt has two functions. When the VUSB is operating in peripheral mode (HOST_MODE_EN=0) this interrupt will detect CRC5 error in the token packets generated by the host. If set the token packet was rejected due to a CRC5 error. When the VUSB is operating in host mode (HOST_MODE_EN=1) this interrupt will detect End of Frame (EOF) error conditions. This would occur when the VUSB is transmitting or receiving data and the SOF counter has reached zero. This interrupt is useful when developing USB packet scheduling software to ensure that no USB transactions cross the start of the next frame.

CRC16 The CRC16 failed.. If set the data packet was rejected due to a CRC16 error.

DFN8 The data field received was not 8 bits. The USB Specification 1.0 specifies that data field must be an integral number of bytes. If the data field was not an integral number of bytes this bit will be set.

BTO_ERR This bit is set if a bus turnaround timeout error has occurred. This VUSB uses a bus turnaround timer to keep track of the amount of time elapsed between the token and data phases of a SETUP or OUT TOKEN or the data and handshake phases of a IN TOKEN. If more that 16-bit times are counted from the previous EOP before a transition from IDLE a bus turnaround timeout error will occur.

DMA_ERR This bit is set if the VUSB has requested a DMA access to read a new BDT but has not been given the bus before VUSB needs to receive or transmit data. If processing an IN TOKEN this would cause a transmit data underflow condition. Or if processing an OUT or SETUP TOKEN this would cause a receive data overflow condition. This interrupt is useful when developing device arbitration hardware for the microprocessor and VUSB to minimize bus request and bus grant latency.

This bit is also set if a data packet to or from the host is larger than the buffer size that is allocated in the BDT. In this case the data packet is truncated as it is put into buffer memory.

RSVD This bit will remain low. It is reserved for future use.

BTS_ERR A bit stuff error has been detected. If set the corresponding packet will be rejected due to a bit stuff error.

8.5 Error Interrupt Enable Register (ERR_ENB 03)

Access: Read\Write

Address: 03\h

Code Reference: err_enb (up_int)

Initialization: 00\h

The Error Interrupt Enable Register (ERR_ENB) contains enable bits for each of the error interrupt sources within the VUSB. Setting any of these bits will enable the respective error interrupt source in the ERR_ISTAT register. This register will contain the value of 00\h after a reset, i.e. all interrupts disabled.

Table 6 ERR_ENB Register Format

7	6	5	4	3	2	1	0
BTS_ERR	RSVD	DMA_ERR	BTO_ERR	DFN8	CRC16	CRC5/EOF	PID_ERR

Error Interrupt Enable Register (ERR_ENB) Bit Definitions:

PID_ERR Setting this bit will enable PID_ERR interrupts.

CRC5/EOF Setting this bit will enable CRC5/EOF interrupts.

CRC16 Setting this bit will enable CRC16 interrupts.

DFN8 Setting this bit will enable DFN8 interrupts.

BTO_ERR Setting this bit will enable BTO_ERR interrupts.

DMA_ERR Setting this bit will enable DMA_ERR interrupts.

RSVD This bit will remain low. It is reserved for future use.

BTS_ERR Setting this bit will enable BTS_ERR interrupts.

8.6 Status Register (STAT 04)

Access: Read Only

Address: 04\h

Code Reference: stat_rg (up_int)

Initialization: None

The Status Register reports the transaction status within the VUSB. When the microprocessor has received a TOK_DNE interrupt, the Status Register should be read to determine the status of the previous endpoint communication. The data in the status register is valid when the TOK_DNE interrupt bit is asserted.

The STAT register is actually a read window into a status FIFO maintained by the VUSB. When the VUSB uses a BD, it updates the status register. If another USB transaction is performed before the TOK_DNE interrupt is serviced the VUSB will store the status of the next transaction in the STAT FIFO. Thus, the STAT register is actually a four byte FIFO which allows the microprocessor to process one transaction while the SIE is processing the next. Clearing the TOK_DNE bit in the INT_STAT register causes the SIE to update the STAT register with the contents of the next STAT value. If the data in the STAT holding register is valid, the SIE will immediately reassert the TOK_DNE interrupt.

Table 7 Status Register Format

7	6	5	4	3	2	1	0
ENDP				IN	ODD	0	0

Status Register (STAT) Bit Definitions:

ENDP3:0	These four bits encode the endpoint address that received or transmitted the previous token. This allows the microprocessor to determine which BDT entry was updated by the last USB transaction.
IN	This bit indicates that the last BDT updated was a was for an IN TOKEN (IN=1), or if IN=0 this bit is 0 the last transaction was an OUT or SETUP TOKEN.
ODD	This bit is indicates that the last buffer descriptor updated was in the odd bank of the BDT, See Section 7.2 For more information on BDT address generation.

8.7 Control Register (CTL 05)

Access: Read\Write

Address: 05\h

Code Reference: ctl_rg (up_int)

Initialization: 00\h

The control register provides various control and configuration information for the VUSB.

7	6	5	4	3	2	1	0
JSTATE	SE0	TxdSuspend TokenBusy	RESET	HOST_ MODE_EN	RESUME	ODD_RST	USB_EN

Name	Description
USB_EN	Setting this bit will enable the VUSB to operate, clearing it will disable the VUSB. Setting this bit causes the SIE to reset all of it's ODD bits to the BDTs. Thus, setting this bit will reset much of the logic in the SIE. When host mode is enabled clearing this bit will cause the SIE to stop sending SOF tokens.
ODD_RST	Setting this bit will reset all the BDT ODD ping/pong bits to 0 which will then specify the EVEN BDT bank.
RESUME	Setting this bit will allow the VUSB to execute resume signaling. This will allow the VUSB to perform remote wake-up. Software must set RESUME to 1 for the required amount of time and then clear it to 0. If the HOST_MODE_EN bit is set the VUSB will append a Low Speed End of Packet to the Resume signaling when the RESUME bit is cleared. For more information on RESUME signaling see Section 7.1.4.5 of the USB specification version 1.0.
HOST_MODE_EN	Setting this bit will enable the VUSB to operate in host mode. In host mode the VUSB will perform USB transactions under the programmed control of the host processor.

RESET	Setting this bit will enable the VUSB to generate USB reset signaling. This will allow the VUSB to reset USB peripherals. This control signal is only valid in host mode, (i.e. HOST_MDOE_EN=1). Software must set RESET to 1 for the required amount of time and then clear it to 0 to end reset signaling. For more information on RESET signaling see Section 7.1.4.3 of the USB specification version 1.0.
TXD_SUSPEND TOKEN_BUSY	This dual use control signal is used to access TXD_SUSPEND when the VUSB is a target, and Token Busy when the VUSB is in host mode. The TXD Suspend bit informs the processor that the SIE has disable packet transmission and reception. Clearing this bit allows the SIE to continue token processing. This bit is set by the SIE when a Setup Token is received allowing software to dequeue any pending packet transactions in the BDT before resuming token processing. The Token Busy bit informs the host processor that the VUSB is busy executing a USB token and no more token commands should be written to the Token Register. Software should check this bit before writing any tokens to the Token Register to ensure that token command are not lost.
SE0	Live USB Single Ended Zero signal.
JSTATE	Live USB differential receiver JSTATE signal. The polarity of this signal is effected by the current state of LS_EN (See the Address Register description below).

8.8 Address Register (ADDR 06)

Access: Read/Write
Address: 06\h
Code Reference: addr_rg (up_int)
Initialization: 00\h

The Address Register (ADDR) contains the unique USB address that the VUSB will decode when in peripheral mode (HOST_MODE_EN=0). When operating in host mode (HOST_MODE_EN=1) the VUSB will transmit this address with a TOKEN packet. This will enable the VUSB to uniquely address any USB peripheral. In either mode the USB_EN bit within the Control register is must be set. The register is reset to 00\h after the reset input has gone active or the VUSB has decoded a USB reset signaling. That will initialize the address register to decode address 00\h as required by the USB specification.

Table 8 Address Register Format

7	6	5	4	3	2	1	0
LS_EN	ADDR6	ADDR5	ADDR4	ADDR3	ADDR2	ADDR1	ADDR0

Address Register (ADDR) Bit Definitions:

ADDR[6:0] This 7-bit value defines the USB address that the VUSB will decode in peripheral mode, or transmit when in host mode.

LS_EN The Low Speed enable informs the VUSB that the next token command written to the token register must be performed at low speed. This will enable the VUSB to perform the necessary preamble required for low speed data transmissions.

8.9 BDT Page Register (BDT_PAGE 07)

Access: Read\Write

Address: 07\h

Code Reference: bdtpg_rg (up_int)

Initialization: 00\h

The Buffer Descriptor Table Page Register contains a 8-bit value that is used to compute the address where the current Buffer Descriptor Table (BDT) resides in system memory. See Section ? for more information on the format of the Buffer Descriptor Table.

Table 9 BDT_PAGE Register Format

7	6	5	4	3	2	1	0
BDT_BA15	BDT_BA14	BDT_BA13	BDT_BA12	BDT_BA11	BDT_BA10	BDT_BA9	BDT_BA8

Buffer Descriptor Table Page Register (BDT_PAGE) Bit Definitions:

BDT_BA[15:8] This 8-bit value is the most significant bits of the BDT base address, which defines where the Buffer Descriptor Table resides at in system memory. The 16-bit BDT base address is always aligned on 256 byte boundaries in memory.

8.10 Frame Number Register (FRM_NUML 08, FRM_NUMH 09)

Access: Read only

Address: 08\h,09\h

Code Reference: fnuml_rg, fnumh_rg (up_int)

Initialization: 00\h

The Frame Number Registers contains the 11-bit frame number. The Frame Number Register requires two 8-bit registers to implement, the low order byte is contained in FRM_NUML and the high order byte is contained in FRM_NUMH. These registers are updated with the current frame number when ever a SOF TOKEN is received.

Table 10 FRM_NUML Register Format

7	6	5	4	3	2	1	0
FRM7	FRM6	FRM5	FRM4	FRM3	FRM2	FRM1	FRM0

Frame Number Low Register (FRML) Bit Definitions:

FRM[7:0] These bits represent the low order bits of the 11-bit Frame Number.

Table 11 FRM_NUMH Register Format

7	6	5	4	3	2	1	0
0	0	0	0	0	FRM10	FRM9	FRM8

Frame Number High Register (FRMH) Bit Definitions:

FRM[10:8] These bits represent the high order bits of the 11-bit Frame Number.

8.11 Token Register (TOKEN 0A)

Access: Read\Write
Address: 0A\h
Code Reference: hc_token_rg (up_int)
Initialization: 00\h

The Token Register is used to perform USB transactions when in host mode (HOST_MODE_EN=1). When the host processor wishes to execute a USB transaction to a peripheral it will write the TOKEN type and endpoint to this register. Once this register has been written the VUSB will begin the specified USB transaction to the address contained in the Address Register. The host processor should always check that the TOKEN_BUSY bit in the control register is not set before performing a write to the Token Register. This will ensure that token commands are not over written before they can be executed. The address register is also used when performing a token command and therefore must also be written before the Token Register. The address register is used to correctly select the USB peripheral address that will be transmitted by the token command.

Table 12 TOKEN Register Format

7	6	5	4	3	2	1	0
TOKEN_PID				TOKEN_ENDPT			

Token Register (TOKEN) Bit Definitions:

TOKEN_PID This 4-bit value is the token type that will be executed by the VSUB. Valid tokens are:

[3:0]

TOKEN_PID	Token Type	Description
0001	OUT Token	VUSB will perform an OUT Transaction
1001	IN Token	VUSB will perform an IN Transaction
1101	SETUP Token	VUSB will perform a SETUP Transaction

TOKEN_ This 4-bit value determines the Endpoint address for the token command. The four bit

ENDPT[3:0] value that is written must be for a valid endpoint.

8.12 SOF Threshold Register (SOF_THLD 0B)

Access: Read\Write
Address: 0B\h
Code Reference: hc_sof_thld_rg (up_int)
Initialization: 00\h

The SOF Threshold Registers are used only in HOST_MODE. When HOST_MODE is enabled, the 14 bit SOF counter counts the interval between SOF frames. The SOF must be transmitted every 1us so the SOF counter is loaded with a value of 12000. When the SOF counter reaches zero, a Start Of Frame (SOF) token is transmitted. The SOF threshold register is used to program the number of usb byte times **before** the SOF to stop initiating token packet transactions. This register must be set to a value that will insure that other packets are not actively being transmitted when the SOF timer counts to zero. When the SOF counter reaches the threshold value, no more tokens will be transmitted until after the SOF has been transmitted. The value programmed into the threshold register must reserve enough time to insure that the worst case transaction will complete. In general the worst case transaction is a IN

token followed by a data packet from the target followed by the response from the host. The actual time required is a function of the maximum packet size on the bus. Typical values for the SOF threshold are: 64 byte packets=74, 32 byte packets=42, 16 byte packets=26, 8 byte packets=18.

Table 13 SOF_THLD Register Format

7	6	5	4	3	2	1	0
CNT7	CNT6	CNT5	CNT4	CNT3	CNT2	CNT1	CNT0

SOF Threshold Register (SOF_THLD) Bit Definitions:

CNT[7:0] These bits represent the SOF count threshold in byte times.

8.13 Endpoint Control Register (EP+CTLn 10-1F)

Access: Read\Write

Addresses: 10\h - 1F\h

Code Reference: endpt0_rg, endpt15_rg (up_int)

Initialization: 00\h

The Endpoint Control Registers contains the endpoint control bits for each of the 16 endpoints available on USB for a decoded address. These four bits define all of the control necessary for any one endpoint. The formats for these registers are shown in **Table 14**. Endpoint 0 (ENDP0) is associated with control pipe 0 which is required by USB for all functions. Therefore, after a USB_RST interrupt has been received the microprocessor should set ENDPT0 to contain 0D\h.

The number of endpoints a design requires can be enabled or disabled in the HDL code. This allows designers to only implement the number of endpoints they require. The file up_int.v(hd) contains these two constants. The constant en_ep_ctl_4_7 when set equal to 1 will force the creation of registers Endpoint control registers 4-7. The constant en_ep_ctl_8_15 when set equal to 1 will force the creation of registers Endpoint control registers 8-15. By default only endpoint registers 0-3 are implemented. See section 13.4 for more information.

Table 14 ENDPTn Register Format

7	6	5	4	3	2	1	0
HOST_WO_HUB	RETRY_DIS	0	EP_CTL_DIS	EP_OUT_EN	EP_IN_EN	EP_STALL	EP_HSHK

End Point Enable Register (ENDPTn) Bit Definitions:

EP_CTL_DIS, EP_OUT_EN, EP_IN_EN

These three bits define if an endpoint is enabled and the direction of the endpoint. The endpoint enable/direction control is defined as follows:

EP_CTL_DIS	EP_OUT_EN	EP_IN_EN	Endpoint Enable/Direction Control
X	0	0	Disable Endpoint
X	0	1	Enable Endpoint for IN tokens only
X	1	0	Enable Endpoint for OUT tokens only
1	1	1	Enable Endpoint for IN and OUT tokens
0	1	1	Enable Endpoint for IN, OUT, and SETUP tokens

EP_STALL When this bit is set it indicates that the endpoint is stalled. This bit has priority over all other control bits in the End Point Enable register, but is only valid if EP_IN_EN=1 or EP_OUT_EN=1. Any access to this endpoint will cause the VUSB to return a STALL handshake. Once an endpoint is stalled it requires intervention from the Host Controller.

EP_HSHK Setting this bit defines if the endpoint will perform handshaking during a transaction to this endpoint. This bit will generally be set unless it is a Isochronous endpoint.

RETRY_DIS This is a host mode only bit and is only present in the control register for endpoint 0 (endpt0_rg). When set this bit causes the host to not retry NAK'ed transactions. When a transaction is NAK'ed the BDT PID field will be updated with the NAK pid, and the token done interrupt will be set. When this bit is cleared NAK'ed transactions will be retried in hardware. This bit must be set when the host is attempting to poll an interrupt endpoint.

HOST_WO_HUB This is a host mode only bit and is only present in the control register for endpoint 0 (endpt0_rg). When set this bit allows the host to communicate to a directly connected low speed device. When cleared the host will produce the PRE_PID then switch to low speed signaling when sending a token to a low speed device as required to communication to a low speed device through a hub.

9 Host Mode Operation

A unique feature of the VUSB core is the optional Host Mode logic. The Host Mode logic allows devices such as digital cameras and palmtop computers to function as a USB Host Controller. Host Mode allows a peripheral such as a digital camera to be connected directly to a USB compliant printer. Digital photos can then be easily printed without having to upload them to a PC. In the palmtop computer application, a USB compliant keyboard/mouse can be connected to the palmtop computer with the obvious advantages of easier interaction.

Host mode is intended for use in handheld-portable devices to allow easy connection to simple HID class devices such as printers and keyboards. It is NOT intended to perform the functions of a full OHCI or UHCI compatible host controller found on PC motherboards. Host mode allows bulk, Isochronous, interrupt and control transfers. Bulk data transfer are performed at nearly the full USB bus bandwidth. Support is provided for ISO transfers, but the number of ISO streams that can be practically supported will be affected by the interrupt latency of the processor servicing the token done interrupts from the SIE. Custom drivers must be written to support Host mode. The VUSB is not supported by Windows 98 as a USB host controller.

The Host Mode logic makes use of the same logic used by the USB function core, and is tightly integrated with the function logic. The `IMPLEMENT_EMBEDDED_HOST` constant defined in the `cfg_usb.vhd` package for VHDL or the `IMPLEMENT_EMBEDDED_HOST` parameter defined within `up_int.v` and `usb_sie.v` for Verilog is used to enable or disable the synthesis of the embedded host logic. When the `IMPLEMENT_EMBEDDED_HOST` constant or parameter is set to 0 (zero), key host signals, and host only registers inputs are forced to zero which allows the logic synthesizer to remove them during logic optimization. If the VUSB core is purchased without the embedded host option. The bulk of the embedded host logic is remove from the source code prior to delivery.

Host Mode is enabled by setting the `HOST_MODE_EN` bit in the CTL register. The VUSB core can only operate as either a Target Device or in Host Mode. It cannot operate in both modes simultaneously. When `HOST_MODE` is enabled, only endpoint zero is used. All other endpoints should be disabled by software. Several additional registers are needed for Host Mode. These registers are signified in the CRB by the GREY color.

9.1 USB Pull-up/pull-down resistors in Host Mode

USB uses pull-up or pull-down resistors to determine when an attach or detach event has taken place on the Bus. Host Mode complicates the issue of these resistors because the device must now operate as either a USB target device or a USB host. Figure 4 shows the two different resistor combinations required for USB targets and hosts. The use of a FET in series with these resistors can be used to dynamically switch from Target to Host modes. The FET in these diagrams can be replaced with a tristate driver on the `HOST_MODE_EN` signal, and a separate tristate signal that is the logical inverse of `HOST_MODE_EN` to

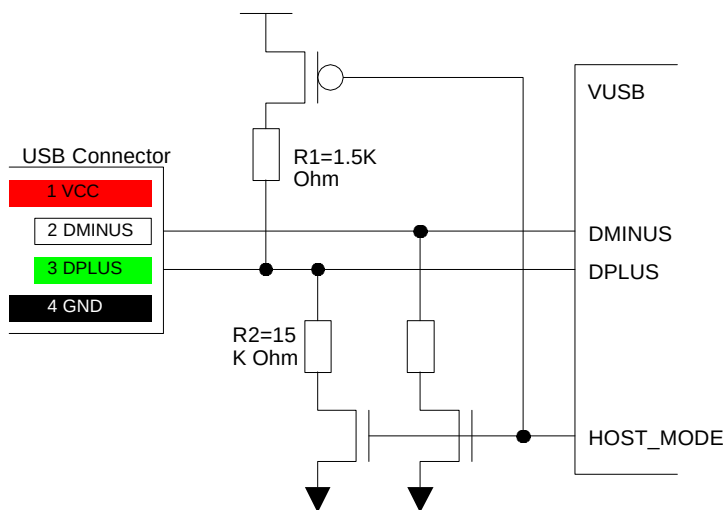


Figure 4

support the target DPLUS pull up resistor. The HOST_MODE_EN bit is provided as a pin on the core for this purpose.

Normally the VUSB operates in normal mode with HOST_MODE_EN=0. In this case, resistor R1 is enabled and the R2 resistors are disabled. When the device is connected to a PC host, the host will recognize that DPLUS is pulled up which indicates an attach by a full speed device.

When the device is in Host Mode (HOST_MODE_EN=1), the R2 resistors are enabled and the R1 resistor is disabled. When a USB target is connected to the VUSB, the R1 in the Target will cause the DPLUS (or DMINUS for a low speed device) signal to go high and the ATTACH interrupt will be activated.

9.2 Sample Host Mode operation

Sample flow charts of some common Host Mode operations are described in this section.

Enable host mode and configure a Target Device:

Step #	Description
1	Enable Host Mode (CTL[HOST_MODE_EN]=1). Pulldown resistors enabled, pullup disabled. SOF generation begins, SOF counter loaded with 12,000.
2	Wait for ATTACH interrupt. Signaled by USB Target pullup resistor changing the state of DPLUS or DMINUS from 0 to 1 (SE0 to J or K state).
3	Enable RESET (CTL[RESET]=1) for 10 ms
4	Host Controller (VUSB) then sends a SETUP token with a GET_DEVICE_DESCRIPTOR packet. An IN token is then issued. Once the data has been received, device type is checked to see if it is compatible with the application. If it is, then another SETUP token is sent with a SET_ADDRESS command followed by a SET_CONFIGURATION command. USB data can then be sent or received from the USB Target device.

High speed bulk data transfer to a Target Device:

Step #	Description
1	Complete all steps to attach and configure the Target device.
2	Write the ADDR register with the address of the target device. Typically there is only one other device on the USB bus in host mode so we would expect the address to be 0x01 and to remain constant.
3	Setup the Even EP0 BDT to transfer up to 64 bytes.
4	Write the TOKEN register with an OUT token to the desired endpoint. The write to this register will trigger the VUSB transmit state machines to begin transmitting the TOKEN and the data.
5	Setup the Odd EP0 BDT to transfer up to 64 bytes.
6	Write the TOKEN register with an OUT token as in step 4. Two Tokens can be queued at a time to allow the packets to be double buffered to achieve maximum throughput.
7	Wait for the TOK_DNE interrupt. This will indicate that one of the BDTs has been released back to the V8 and that the transfer has completed. If the Target device asserts NAKs, the VUSB will continue to retry the transfer indefinitely without processor intervention. If a Stall interrupt occurs, then the pending packet must be dequeued and the error condition in the Target device cleared. If a RESET interrupt occurs (SE0 for more than 2.5us), then the target has detached and Host Mode should be exited.
8	Once the TOK_DNE interrupt occurs, the BDTs can be examined and the next data packet queued by returning to step 2.

10 Signal Descriptions

Pin Name	Direction	Description
ADDRI[15:0]	IN	Microprocessor Address Bus
ADDRO[15:0]	OUT	Microprocessor Address Bus
DMADATAI[7:0]	IN	DMA Data Bus In
DATAI[7:0]	IN	Microprocessor Data Bus In
DATAO[7:0]	OUT	Microprocessor Data Bus Out
DOE	OUT	Data Bus Output Enable
CEI	IN	Chip Enable Input
CEO	OUT	Chip Enable Output
WEI	IN	Write Enable Input
WEO	OUT	Write Enable Output
REI	IN	Read Enable Input
REO	OUT	Read Enable Output
RDYI	IN	Bus Ready Input
RDYO	OUT	Bus Ready Output

Pin Name	Direction	Description
BUS_REQ	OUT	Bus Request
BUS_GNT	IN	Bus Grant
BMOE	OUT	Bus Master Signals Output Enable
BSOE	OUT	Bus Slave Signals Output Enable
INT	OUT	Interrupt
RESET	IN	Reset
CLK	IN	Clock Input
SPEED	OUT	USB Speed Indicator
SUSPND	OUT	USB Suspend
USBOE	OUT	USB Output Enable
DPO	OUT	USB Data Plus Output
DMO	OUT	USB Data Minus Output
RCV	IN	USB Receive Data
EOP	IN	USB End Of Packet
SE0	IN	USB Single Ended Zero
HOST_MODE	OUT	HOST Mode Enable

10.1 Generic Microprocessor Bus Interface Signals

The VUSB provides a generic microprocessor bus interface. As a slave on the bus the VUSB allows access to its internal control registers. The asynchronous DMA controller within the core can perform bus master cycles to access the buffer descriptor tables and transfer USB packet data to and from memory.

Address Input Bus (ADDRI[15:0])

The address input bus (ADDRI) is used in conjunction with the CEI signal to identify the target of slave accesses to or from the VUSB core.

Address Output Bus (ADDRO[15:0])

In master mode the DMA interface within the VUSB core drives RAM addresses out on the ADDRO bus.

DMA Data Input Bus (DMADATAI[7:0])

In master mode the DMA data input bus is used to transfer data from memory to the VUSB core for master read operations.

Data Input Bus (DATAI[7:0])

The data input bus DAT

AI is used to drive data into the VUSB core for slave write operations.

Data Output Bus (DATAO[7:0])

The data output bus DATAO is used for slave read operations. During slave read operations the data is guaranteed to be valid when the RDYO line is asserted.

Data Bus Output Enable (DOE)

The data bus output enable should be used as an output enable signal when the data bus signals are connected to bidirectional tristate drivers. When the DOE signal is active the DATAO signal should be driven onto the bus.

Chip Enable Input (CEI)

The chip enable input, when driven active, indicates a slave access to the VUSB internal registers. CEI is used in conjunction with WEI, and REI to determine if a slave write or read operation is being performed. The ADDR[4:0] are used to select the internal register within the VUSB.

Chip Enable Output (CEO)

The chip enable output is driven active by the VUSB core when the DMA controller within the core is accessing RAM as a bus master.

Write Enable Input (WEI)

The write enable input, when active, indicates a slave write to the VUSB core. ADDR[4:0] and CEI must be active for the write to be valid.

Write Enable Output (WEO)

The write enable output signal is used during bus master cycles by the DMA controller to signify a bus master write cycle.

Read Enable Input (REI)

The read enable input, when active, indicates a slave read to the VUSB core. ADDR[4:0] and CEI must be active for the read to be valid.

Read Enable Output (REO)

The read enable output signal is used during bus master cycles by the DMA controller to signify a bus master read cycle.

Ready Input (RDYI)

The ready input is used to indicate that a bus master read or write operation requested by the VUSB core can be completed. The RDYI input is sampled on the rising edge of CLK.

Ready Input (RDYO)

The ready output is used to indicate that a slave read or write operation from or to the VUSB core has been completed. In the case of read operation it indicates that the read data on the bus is valid.

Bus Request (BUS_REQ)

The bus request output indicates that the VUSB core would like control of the bus to perform a bus master read or write operation.

Bus Grant (BUS_GNT)

The bus grant input indicates that the VUSB core has been granted control of the bus.

Bus Master Output Enable (BMOE)

The bus master output enable is used to control the tristate drivers of bus signals used when the VUSB core is the bus master. These signals are ADDRO, CEO, WEO and REO. In an embedded application that does not use tristate busses the BMOE signal can be ignored, or could be used to control the multiplexor select lines for these signals.

Bus Slave Output Enable (BSOE)

The bus slave output enable is used to control the tristate drivers of bus signals used when the VUSB core is accessed as a bus slave. The signal controlled by BSOE is RDYO. In an embedded application that does not use tristate busses the BSOE signal can be ignored, or could be used to control the multiplexor select lines for the RDY signal.

Interrupt (INT)

The interrupt (INT) output is used to signal events to the microprocessor. The state of the interrupt pin is defined by the combination of the interrupt status register and the interrupt mask register as defined in the register definition section.

Reset (RESET)

The reset input signal is used to reset the VUSB core. It must be held high for at least 4 CLK intervals to insure proper initialization. RESET is synchronized by the rising edge of CLK before it is used internally. Upon reset activation all registers are initialized to the value shown in the register definition section. In addition any ongoing transfers are aborted.

10.2 USB Interface Signals

The VUSB core provides a digital interface for USB related signals. This allows the design extra flexibility when implementing USB design solutions. This interface can be used to implement an on-chip USB transceiver provided your target technology can support USB I/Os or it can be directly connect to a Philips Semiconductor USB transceiver (Part# PDIUSBP11).

Clock (CLK)

The clock input is required to be connected to a 12 MHz signal that is derived from the USB signals. The clock input is designed to be connected to the clkout signal from the dpllNrzi.v(hd). For more information on connecting the Digital Phase Lock Loop to the VUSB see the sie.v(hd) file.

USB Speed Indicator (SPEED)

The USB speed indicator is used by external USB transceiver logic to determine which speed interface the VUSB is implementing. When SPEED = 1 the VUSB is operating as a high speed USB device, when SPEED = 0 the VUSB is a low speed device.

USB Suspend (SUSPND)

The USB suspend signal is used by external logic to determine when the VUSB is in suspend mode. This is useful when external logic must enter a low power mode during suspend. When SUSPND = 1 the VUSB is suspended, when SUSPND = 0 the VUSB is operational.

USB Output Enable (USBOE)

The USB output enable signal is designed to be connected to the tristate control of USB transceivers. When active, USBOE = 1, the VUSB core will drive serial data on to the USB.

USB Data Plus Output (DPO)

The USB data plus output signal is used to transmit the NRZI encoded serial data to the D+ side of the USB.

USB Data Minus Output (DMO)

The USB data minus output signal is used to transmit the NRZI encoded serial data to the D- side of the USB.

USB Receive Data (RCV)

The USB receive data input should be connected to a NRZ serial data stream decoded from the USB D+ and D- signals. Typically this signal will be connected to DATAOUT output from the digital phase lock loop, dpllNrzi.v(hd). For more information on connecting the digital phase lock loop to the VUSB see the sie.v(hd) file. The VUSB core assumes that this input signal is synchronous to the CLK signal. Designers not using the supplied digital phase lock loop, dpllNrzi.v(hd), should be aware of this.

USB End Of Packet (EOP)

The USB end of packet input should be active when a end of packet condition has been decoded on the USB D+ and D- signals. Typically this signal will be connected to EOP output from the digital phase lock loop, `dp1lnrz.v(hd)`. For more information on connecting the digital phase lock loop to the VUSB see the `sie.v(hd)` file. The VUSB core assumes that this input signal is synchronous to the CLK signal. Designers not using the supplied digital phase lock loop, `dp1lnrzi.v(hd)`, should be aware of this.

USB Single Ended Zero (SE0)

The USB single ended zero input should be active when a single ended zero condition has been decoded on the USB D+ and D- signals. Typically this signal will be connected to SE0 output from the digital phase lock loop, `dp1lnrz.v(hd)`. For more information on connecting the digital phase lock loop to the VUSB see the `sie.v(hd)` file. The VUSB core assumes that this input signal is synchronous to the CLK signal. Designers not using the supplied digital phase lock loop, `dp1lnrzi.v(hd)`, should be aware of this.

HOST Mode Enable (HOST_MODE)

The HOST Mode Enable signal is used to provide external programmable control of Host Mode functions. This would typically include the pull-up/pull-down resistors needed to implement a USB target peripheral or a USB Host controller. For more information on the required pull-up/pull-down control see Section 9.1 for more information.

11 Bus Cycles

The VUSB supports a master and slave interface. This translates to 4 different types of bus cycles. Each is described in detail below, for additional timing information see Section 12.

11.1 VUSB Slave Write Bus Cycle

To write the VUSB internal registers the system microprocessor must perform a slave write bus cycle. This bus cycle is distinguished by the CEI and WEI being active while REI is inactive. The target VUSB register is based on the decoded address present on the bus. The write data is written on the rising edge of clock. Therefore ADDRI, CEI, WEI, REI, and DATAI must meet setup and hold timing to the rising edge of clock. In a fully synchronous designs this bus cycle requires a single clock cycle to complete. This is shown in Figure 5.

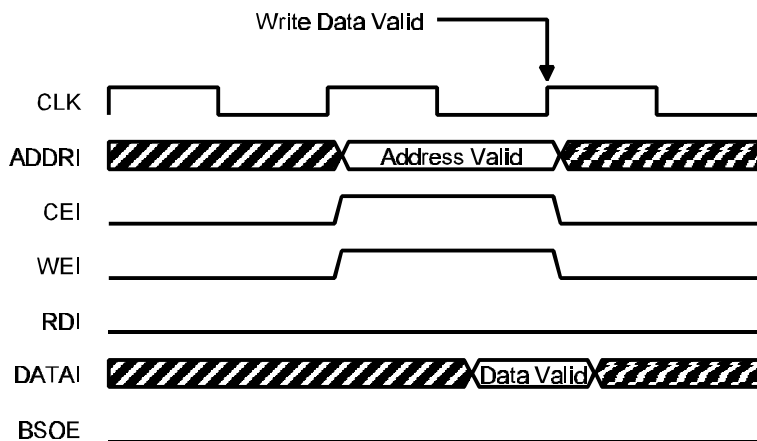
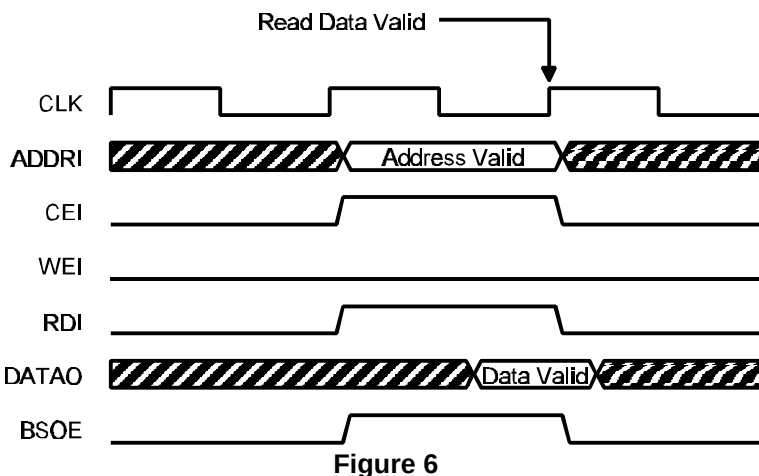


Figure 5

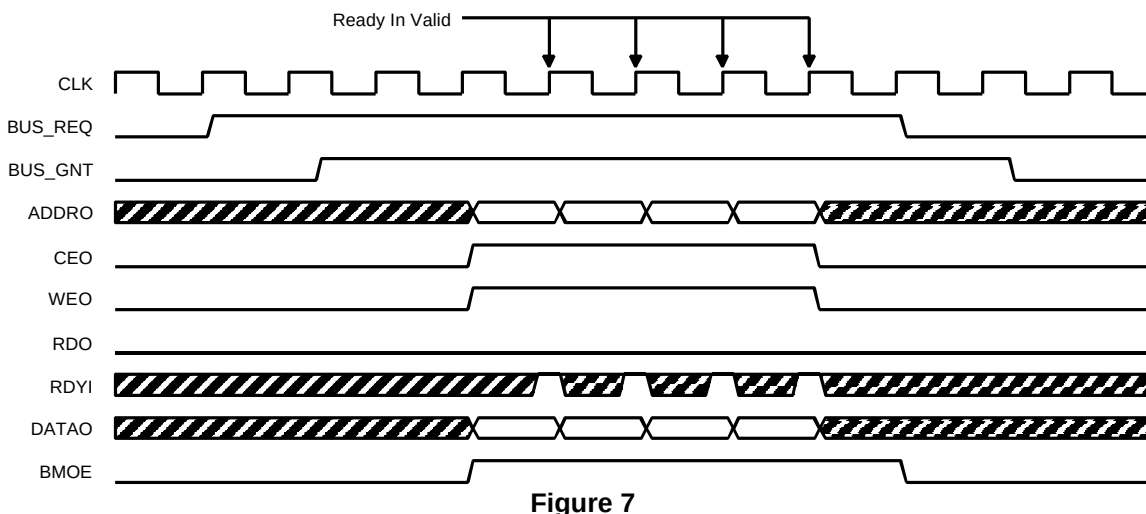
11.2 VUSB Slave Read Bus Cycle

To read the VUSB internal registers the system microprocessor must perform a slave read bus cycle. This bus cycle is distinguished by the CEI and REI being active while WEI is inactive. The target VUSB register is based on the decoded address present on the bus. The read data is output on DATAO bus and will be valid before the rising edge of clock. The BSOE controls the output enable for the DATAO drivers and is the decode of CEI and REI both being active. In a fully synchronous designs this bus cycle requires a single clock cycle to complete. This is shown in Figure 6.



11.3 VUSB Master Write Bus Cycle

To transfer USB data fast and efficiently the VUSB also supports bus master capabilities. When the VUSB must write data to memory it will perform a master write bus cycle. This type of bus cycle starts with the VUSB requesting the bus by asserting the BUS_REQ signal. The VUSB will then sample the BUS_GNT line on every rising edge of CLK. When active, the VUSB will begin a master write bus cycle by asserting the BMOE signal. The BMOE signal is typically used to control tri-state buffers and is active the entire time the VUSB is master of the bus. Address is output on ADDRO, write data on DATAO, and control signals CEO, WEO will be active while REO is inactive. The RDYI input is sampled on the rising edge of CLK following an address change and is used to add additional wait states if necessary. The signals ADDRO, DATAO, CEO, WEO, REO, and BMOE are from the outputs of flip-flops for clean signal transitions. In a fully synchronous designs these bus cycles requires a single clock cycle to complete. This is shown in Figure 7.



11.4 VUSB Master Read Bus Cycle

To transfer USB data fast and efficiently the VUSB also supports bus master capabilities. When the VUSB must read data to memory it will perform a master read bus cycle. This type of bus cycle starts

with the VUSB requesting the bus by asserting the BUS_REQ signal. The VUSB will then sample the BUS_GNT line on every rising edge of CLK. When active, the VUSB will begin a master read bus cycle by asserting the BMOE signal. The BMOE signal is typically used to control tri-state buffers and is active the entire time the VUSB is master of the bus. Address is output on ADDRO, and control signals CEO, REO will be active while WEO is inactive. The RDYI input is sampled on the rising edge of CLK following an address change and is used to add additional wait states if necessary. The DATAI inputs are also sampled on the rising edge of CLK following an address change and are assumed valid when RDYI is also true. The signals ADDRO, CEO, WEO, REO, and BMOE are from the outputs of flip-flops for clean signal transitions. In a fully synchronous designs these bus cycles requires a single clock cycle to complete. This is shown in Figure 8.

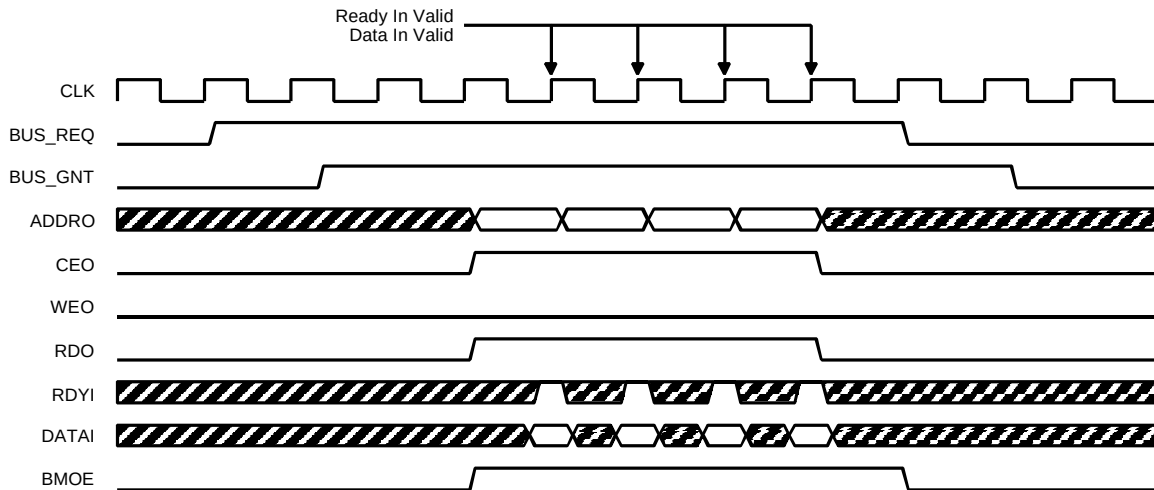


Figure 8

12 Timing

This section describes the signal timing for the VUSB core. To alleviate clock tree buffering problems a single clock input (CLK) is used on the VUSB core. Designers should insert a clock tree buffering scheme on this input. This section only documents actual signal timing for a more detail description of the actual bus cycles see Section 11. All inputs on the VUSB must observe a setup and hold time relative to the rising edge of clock. See **Figure 9**. The following list of signals are all the inputs that must adhere to this input timing:

ADDRI(15:0), DMADATAI(7:0), DATAI(7:0), CEI, WEI, REI, RDYI, BUS_GNT, RESET, RCV, EOP, SE0

For more details on any of these signals see Section 10.

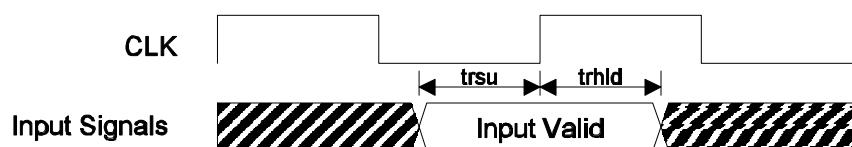


Figure 9 Input Timing

Most signals are directly from the output of a flip flop. This ensures outputs have a clean transition.

Since this is a synthesizable core and not an IC, actual timing parameters cannot be provided. Timing parameters will vary significantly depending on the synthesizer used and the target technology, and provide timing parameters for your reference. The following chart also provides some relative delay times available when the core was synthesized to a 0.8 micron CMOS technology. Use these numbers only as a guide. Your actual delay times may vary significantly. There is an extra column in the chart where you can include the delays from your implementation of the VUSB.

13 Synthesizing the VUSB

This section is intended to assist customers when synthesizing the VUSB. It is assumed the user has a understanding of their synthesizers operation and this discussion is not tailored to any particular synthesizer. The VUSB is shipped in either VHDL or Verilog source code format. The HDL code is hierarchical which allows the designer additional flexibility when synthesizing. A single file can be synthesized at a time, or any number can be synthesized together. Section 13.2 describes the file hierarchy for the VUSB core. Generally the best results are achieved when the entire design is synthesized together.

The VUSB design is static and full synchronous to make timing and synthesis simple. The DPLL, however does contain three negative edge triggered flip-flops to gain additional resolution on the position of an incoming transition relative to the current DPLL clock. Because these negative edge flops may cause a problem for automatic test insertion or other areas of your tool chain we have grouped the logic together so that it maybe easily removed from your design. Refer to the comments in the dpllri.v(hd) file for additional information on the connection or removal of these flip-flops as well as a discussion of their effect on the DPLL performance .

The design flow is simple and straight forward. The designer simply selects the configuration of the

VUSB and then read the files into the synthesizer. The designer can specify whether to optimize for speed or area, set a maximum delay, and other user constraints specific to the synthesizer. A target technology is also selected. The results from the synthesis should then be run through a static timing analyzer. This program will check the delays through all nets in the design to make sure that it can operate at your designated frequency. If any nets did not make timing, synthesis can be rerun with additional user constraints on the slow nets. This generally will continue until all timing has been satisfied. This design flow is shown in **Figure 10**.

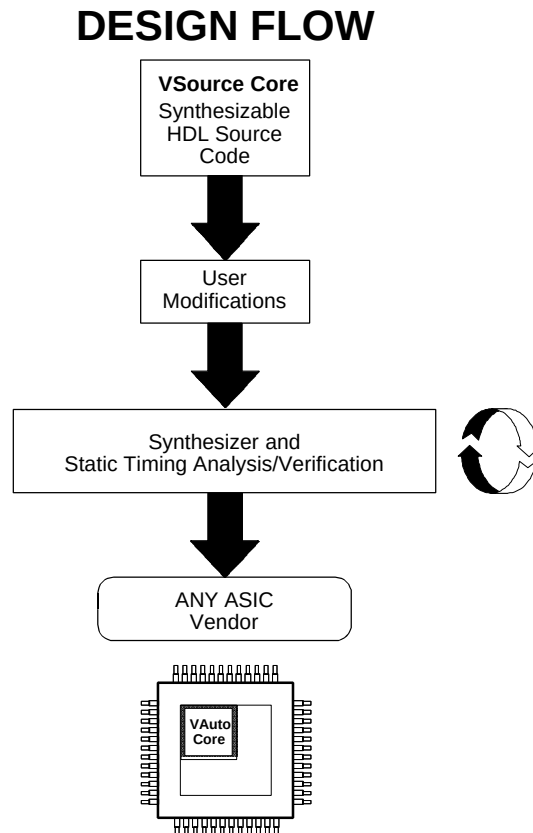


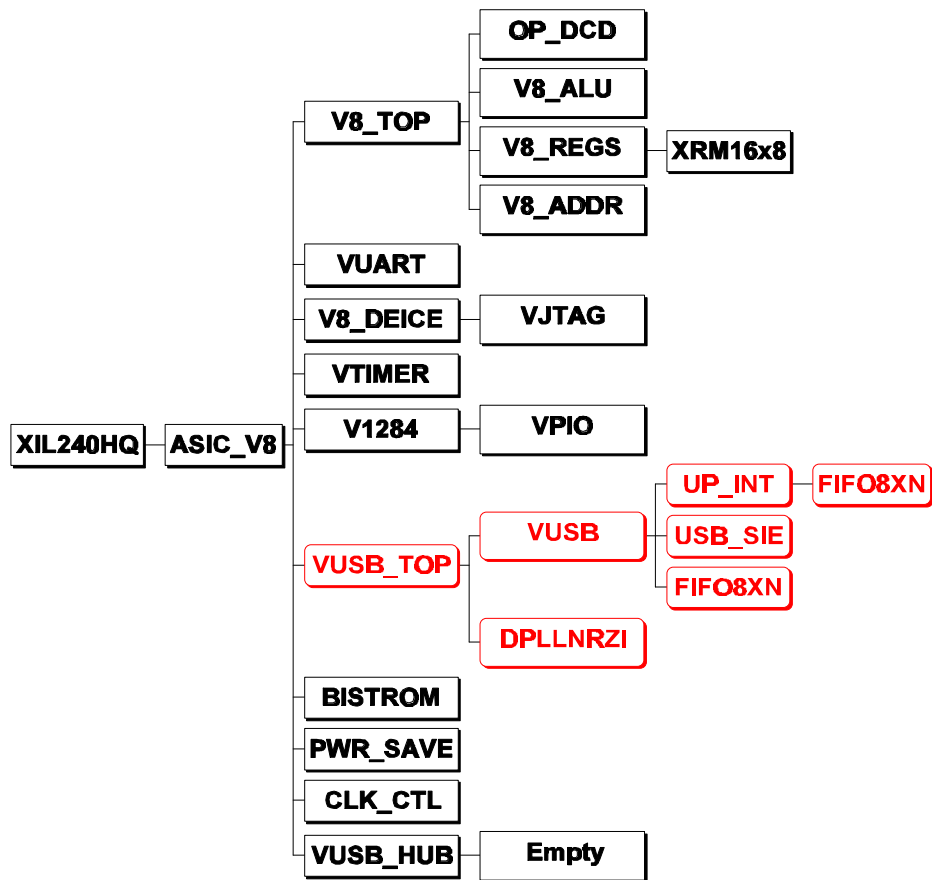
Figure 10 Design Flow

13.1 Makefile

The Makefile is a UNIX makefile which can be used and easily modified to compile or synthesize the model. File dependencies are included so that when analyzing for simulation or synthesizing, only the files that have changed will be reanalyzed or resynthesized. The Makefile supports various synthesizers and simulators and may require some environment variables to be setup in order for it to operate properly. See the comments in this file for more information.

13.2 VUSB File Hierarchy

The highlighted files in the drawing below indicate the files associated with the VUSB core. The other files are the various peripherals and other logic used to create the V8 based USB



microcontroller used in the IntelliCore Prototype System. Each file of the VUSB core is briefly described below. See the header inside each file for more detail.

File Name	Description
XIL240HQ.V(HD)	Top level structural logic that creates the input, output and tristate IO cells. Clock buffering cells and the external RAM write enable logic is included here as it often requires special placement and timing constraints.
ASIC_V8.V(HD)	Top Level mostly structural module which interconnects all of the V8 peripherals to the V8. Some logic is created at this level for various control registers, readback muxes and address decoding.
V8_TOP.V(HD)	Top level module for the V8-uRISC CPU
OP_DCD.V(HD)	V8 Opcode decoding logic
V8_ALU.V(HD)	V8 Arithmetic Logic Unit

File Name	Description
V8_REGS.V(HD)	16x8 dual port register file. Two architectures are provided, SYNTH and XILINX. SYNTH will result in DFF/MUX implementation of about 1700 gates, XILINX is ideal for Xilinx FPGAs and results in 8 CLBs instead of over 64.
V8_ADDR.V(HD)	Address generation logic
VUART.V(HD)	8 bit UART with integrated BAUD rate generator
VJTAG.V(HD)	IEEE 1149.1 JTAG TAP controller
VTIMER.V(HD)	16 Bit counter/timer
V1284.V(HD)	IEEE 1284 Parallel Printer port
VPIO.V(HD)	8 bit Programmable Parallel IO
VUSB_TOP.V(HD)	VUSB top level of the VUSB core including DPLL clocking logic.
VUSB.V(HD)	VUSB integrates microprocessor interface and USB serial interfaces engine.
DPLLRZI.V(HD)	Digital phase lock loop with NRZ serial data output.
UP_INT.V(HD)	Microprocessor interface.
USB_SIE.V(HD)	USB serial interface engine.
FIFO8XN.V(HD)	Byte wide by n deep synchronous fifo.
CLK_CTL.V(HD)	Mostly interconnect to direct the appropriate clock and reset signals to the other blocks based on the architecture selected.
PWR_CTL.V(HD)	Small block that can be used to save power by halting the processor or stopping the global clock under processor control. This block contains asynchronous resets.
BISTROM.V(HD)	512x8 ROM behavioral model

13.3 VUSB Configurations

The VUSB core is designed to fit a wide variety of USB applications. To meet these different demands, there are four different configurations are provided with the VUSB. Two configurations are provided for the VUSB Target Architecture, and two configurations are provided for the VUSB Host Controller Architecture. For more information on the VUSB Architectures see Section 14.1.

The two configurations are; Asynch, and High_Performance. These different configurations primarily select the clocking structure desired in the VUSB core. This allows the designer different power/performance trade-offs based on the end USB application. For Target implementations the two configurations are a_v8_usb (i.e. Asynch), and a_v8_usb_hi_perf (i.e. High_Performance). And for Host Controller implementations the two configurations are a_v8_usb_hc (i.e. Asynch), and a_v8_usb_hc_hi_perf (i.e. High_Performance). Each configuration has it's benefits and features as

shown in the table below:

Configuration	Description	Sample Applications
Asynch: (a_usb_hsdev_async for targets) (a_usb_hc_hsdev_async for host controllers) (a_usb_hc_lsdev_async for host controllers connecting to a low speed device)	Microprocessor Interface (UP_INT) operates on the 12 MHz clock asynchronously to the microprocessor.	Ideal for most applications with light CPU requirements and especially when using FPGAs. The microprocessor clock is a constant 12 MHz. Bus cycles are lost due to the asynchronous nature of the bus.
High Performance: (a_usb_hsdev_sync for targets) (a_usb_hc_hsdev_sync for host controllers) (a_usb_hc_lsdev_sync for host controller communicating to a low speed device)	Microprocessor Interface operates at the DPLL clock of 48 Mhz and is assumed to be synchronous to the microprocessor.	Ideal for applications needing significant performance from the V8-uRISC. USB requirements on the memory bus are minimized. Power consumption may be somewhat high since the microprocessor is running at 48 MHz.

NOTE for Verilog users:

Verilog does not support the concept of multiple configurations or architectures for files of the same name. To provide the same functionality available in VHDL, separate files are provided for each configuration or architecture. For example the VHDL file VUSB_TOP.VHD contains three architectures: empty, async, and sync. To implement each of these architectures in Verilog a different file is created. The file VUSB_TOP_EMPTY.V implements the empty architecture, VUSB_TOP_ASYNC.V implements the async architecture, and VUSB_TOP_SYNC.V implements the sync architecture. So to implement one of the different configurations above, a Verilog user needs to load the corresponding architecture files for that configuration. For simulation the standard configuration are all taken care of for the Verilog user, See Section 14.5 for more information.

13.4 VUSB Programmable Control

The VUSB has the ability to be programed for different modes of operation. These modes can be programmed by changing the values in the CFG_USB.V(HD) file. To do this the user only has to modify the values of the signals for VHDL or parameters for Verilog. Below is a brief description of each of these programmable values follows:

- LOW_SPEED_DEVICE is used by the DPLLNRZI module in the testbench
- host controller to determine the polarity of the dplus and dminus
- signals for the J, K and SE0 USB states. When set to 1 it
- configures the logic as a low speed device:
- K state = {dminus low, dplus high}
- J state = {dminus high, dplus low}
- Note that for a low speed device the input clock (usbclkx4) needs
- to be 6MHz.
- When set to 0 it configures the logic for a high speed device.
- K state = {dminus high, dplus low}
- J state = {dminus low, dplus high}
- Note that for a high speed device the input clock (usbclkx4) needs

```

-- to be 48MHz.
CONSTANT LOW_SPEED_DEV      : std_logic := '0'; -- high speed device
-- The LOW_SPEED_DEV is separate from the low_speed_en signal that allows
-- A full speed host or target to take on the signalling of a low speed
-- device dynamically.

-- IMPLEMENT_EMBEDDED_HOST is used within the VUSB core in the usb_sie, and the
-- up_int to automatically allow or remove gates associated with the embedded
-- host controller function. When this constant is set to '1' the host
-- embedded host controller functions are implemented. When set to '0' this
-- constant will cause most of the gates associated with the embedded host
-- function to be removed by your synthesizer.
CONSTANT IMPLEMENT_EMBEDDED_HOST : std_logic := '1'; -- implement embedded host

-- HOST_WITHOUT_HUB is used within the DPLLNRZI module to determine the
-- signalling polarity of the USB J and K states. When this constant is set
-- to '1' a host that is transmitting a low speed packet directly to a low
-- speed device without an intervening hub will invert the J/K state polarity
-- of the DPLUS and DMINUS signals.
CONSTANT HOST_WITHOUT_HUB : integer := 0;

-- These two constants control the number of endpoints the up_int will
-- implement. When set to zero the endpoints won't be implemented.
-- When set to 1 the endpoints will be implemented.
CONSTANT EN_EP_CTL_4_7 : std_logic := '0'; -- Enables endpoints 4-7
CONSTANT EN_EP_CTL_8_15 : std_logic := '0'; -- Enables endpoints 8-15

-- This constant is used to enable the DMA controller within the
-- uProcessor interface to write the entire BDT back to memory.
-- Normally the DMA controller will only write back the first two
-- bytes of the BDT when EN_BDT_4_BYTE_WRITE equals 0. When set to 1
-- the DMA controller will write back the entire four bytes of the BDT.
CONSTANT EN_BDT_4_BYTE_WRITE : std_logic := '0';

-- This constant is used within the uProcessor interface block to
-- determine whether the interface requires clock enable
-- synchronization logic.

-- In an FPGA device this constant would be set to 0 and only the
-- slv_acc_tgl and ready_tgl signals would need to meet the 48MHz
-- operation.

-- In an ASIC implementation this constant is set to 1 and the clock
-- enable synchronization logic is removed under the assumption that
-- all the register to register logic will be timed at 48MHz, and
-- that the uProcessor will be running on the 48MHz clock.
CONSTANT ASIC_IMPLEMENTATION : std_logic := '0';

```

en_ep_ctl_4_7 - Enables the creation of endpoint control registers 4 through 7. If a designer wants to build a design that implements these endpoints set en_ep_ctl_4_7 = 1, to disable set en_ep_ctl_4_7 = 0. The default value is en_ep_ctl_4_7 = 0.

en_ep_ctl_8_15 - Enables the creation of endpoint control registers 8 through 15. If a designer wants to build a design that implements these endpoints set en_ep_ctl_8_15 = 1, to disable set en_ep_ctl_8_15 = 0. The default value is en_ep_ctl_8_15 = 0.

14 Test Bench Description

A test bench is supplied along with the VUSB synthesizable modules. The test bench is provided in source code format. The test bench is a system level simulation environment that emulates the Intellicore™ Prototyping System.

The key to using the test bench effectively is to build a complete system-level simulation environment of your system. A simulation environment enables you to debug your ASIC before you have committed the design to silicon. You can also integrate and debug software in the HDL simulator and make hardware/software design tradeoffs. The test bench provided is an example of the Intellicore™ Prototype System (IPS) which is available from VAutomation.

There are four primary files used for controlling the test bench, HOST_CTL.V(HD), IPS1_TRM.DAT, HOST_TRM.DAT, BISTROM.HEX, and RAM128k8.HEX files. The HEX files are replaced with MEM files for Verilog users. The HOST_CTL.V(HD) file is a Behavioral Host Controller. It can execute an entire suite of USB tests. These tests can be used to verify compliance of your USB device. For more information on the Behavioral Host Controller and its API see Section 14.3. The IPS1_TRM.DAT and HOST_TRM.DAT files simulate commands entered from a terminal connected to the RS-232 port of the IPS. With this port you can enter commands just as you would in the real IPS. Also, any debugging information that would normally be transmitted out the RS-232 port is echoed to the screen during simulation. The IPS1_TRM.DAT is connected to the target IPS and simulates commands that would be sent and received to and from the target IPS. The HOST_TRM.DAT is connected to the host controller IPS and simulates commands that would be sent and received to and from the host controller IPS. HOST_TRM.DAT is only used if one of the architectures that implements the VUSB Host Controller is loaded. Finally, the HEX files are Intel Hex files output from the V8 C-Compiler or V8ASM assembler. These files are loaded in to RAM models during simulation. This is how you get your programs into the simulator for the V8 to execute. The RAM128k8.HEX file is loaded into the RAM128k8.VHD model. For Verilog users RAM128k8.HEX is converted to RAM128k8.MEM and loaded into RAM128k8.V module. The BISTROM.HEX file is loaded into the BISTROM.VHD model. For Verilog users BISTROM.HEX is converted to BISTROM.MEM and loaded into BISTROM.V module. The conversion of the Intel hex files to Verilog MEM files is performed with the ihx2v program. Note that if you want to replace the BISTROM you can do that easily by changing the BISTROM.HEX file.

14.1 VUSB Test Bench File Hierarchy and Architectures

There are numerous files, architectures, and configurations associated with the VUSB test bench. The VUSB Target Architecture is shown in **Figure 12**. This architecture implements the TESTBNCH.V(HD) module at the top level and simply contains a small amount of structural logic to interconnect HOST_CTL.V(HD), IPS_PCB.V(HD), and VPC_TERM.V(HD) modules. The VUSB Host Controller Architecture is shown in **Figure 13**, is slightly different than the target architecture. This architecture implements the TESTBNCH.V(HD) module at the top level and again contains a small amount of structural logic to interconnect two IPS_PCB.V(HD) modules, and two VPC_TERM.V(HD) modules. The host controller is implemented with the IPS_PCB.V(HD) module labeled UHOST_CTL. It is connected to the VPC_TERM.V(HD) module labeled HOST_TERM and reads commands from the HOST_TRM.DAT file. The target device is implemented with the IPS_PCB.V(HD) module labeled IPS1. It is connected to the VPC_TERM.V(HD) module labeled IPS1_TERM and reads commands from the IPS1_TRM.DAT file.

In both target and host controller architectures of the VUSB, the TESTBNCH.V(HD) module instantiates the IPS_PCB.V(HD). The IPS_PCB.V(HD) module is a structural model that was used to build the Intellicore™ Prototyping System (IPS) printed circuit board. The IPS_PCB.V(HD) module calls out many small modules that have been placed in the PCB_LIB subdirectory for simplicity. All of the modules in the PCB_LIB subdirectory are behavioral and are only used for simulation purposes. Most of the modules

are actually empty and are needed as place holders to match the components on the PCB.

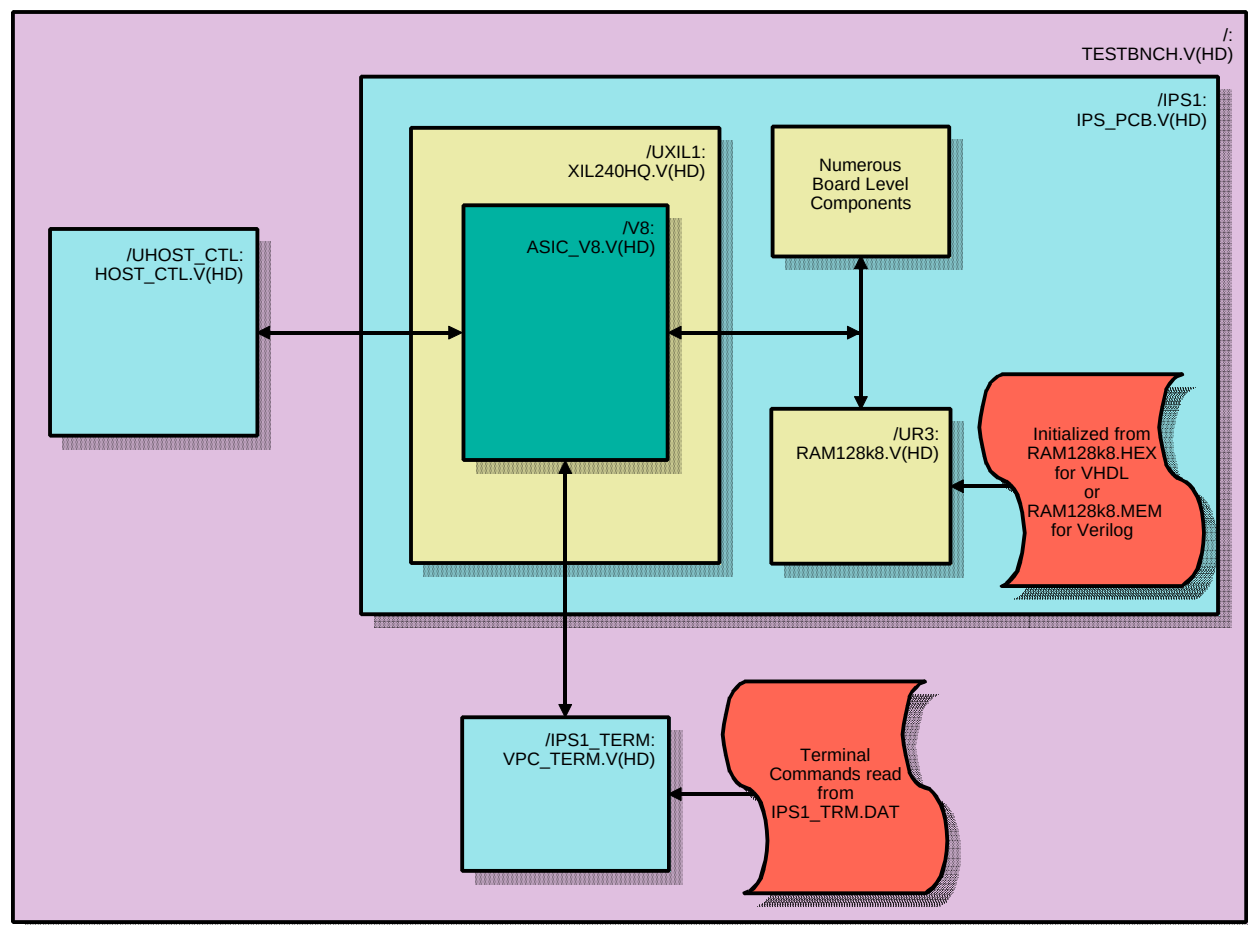


Figure 12 VUSB Target Test Bench Architecture

The IPS_PCB.V(HD) module instantiates the XIL240HQ.V(HD) module which in turn calls out the ASIC_V8.V(HD) module. Within the ASIC_V8.V(HD) module is where the synthesizable VUSB, V8-uRISC, and associated V8 peripheral cores are implemented. The ASIC_V8.V(HD) module gives the user an excellent example of how to implement an ASIC design based on the VUSB and V8 μ RISC™. V8 specific files have been placed in the V8_LIB subdirectory, and HUB specific files have been placed in the HUB_LIB subdirectory. Using these subdirectories avoids additional files in the top level VUSB directory. Here is brief description of the files mentioned. See the headers in the files for more detail.

File Name	Description
TESTBNCH.V(HD)	Top level structural module. The VHDL version contains all of the various VUSB configurations.
VPC_TERM.V(HD)	Reads the VPC_TERM.DAT file and transmits the commands into the V8-uRISC RS-232 port. Observes the RS-232 transmit channel and echoes any characters to the simulator transcript window.
IPS_PCB.V(HD)	Structural model of the IntelliCore™ Prototype System printed circuit board.

File Name	Description
RAM128K8.V(HD)	128Kx8 behavioral RAM model that can read Intel hex files output by the V8ASM assembler. The input filename is RAM128K8.HEX for VHDL and RAM128K8.MEM for Verilog.
XIL240HQ.V(HD)	Implements a Xilinx version of ASIC_V8. All I/O assignments are implements at this level.
ASIC_V8.V(HD)	Top level of the V8 MicroController complete with VUSB, and all V8 peripherals.

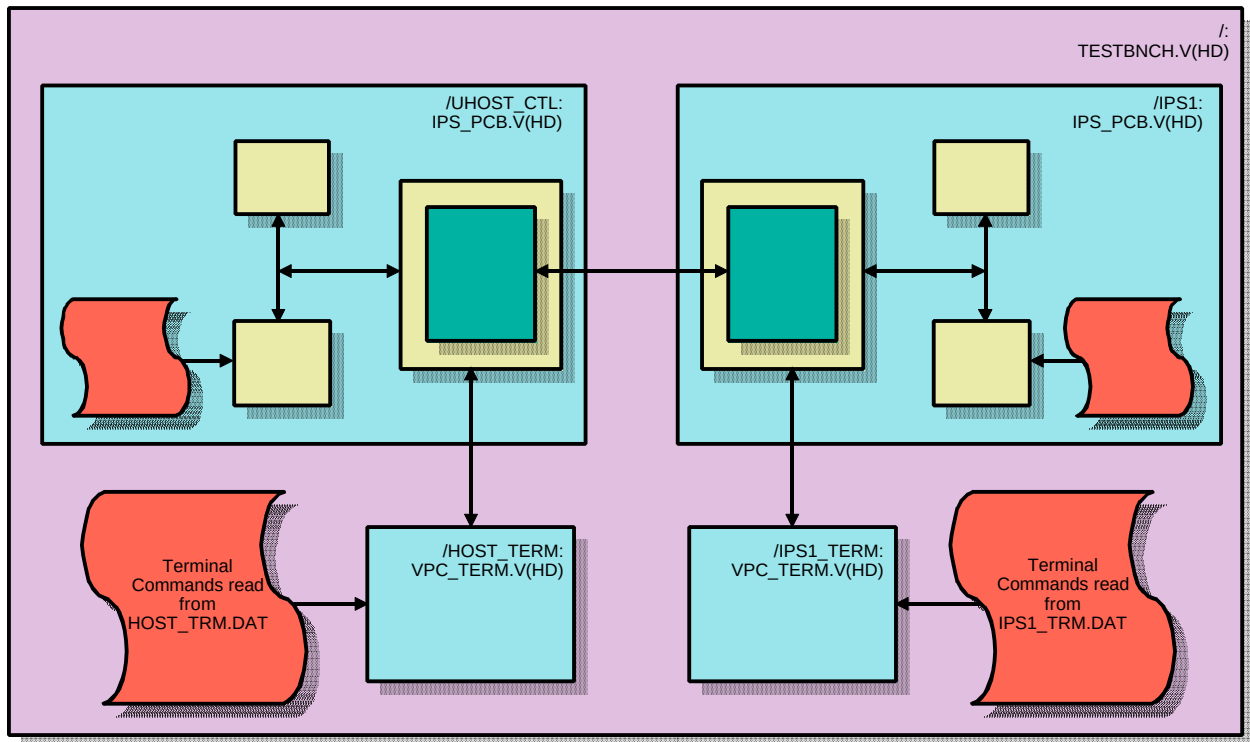


Figure 13 VUSB Host Controller Test Bench Architecture

14.2 Programmable Control of the Test Bench

The VUSB Test Bench is fully self contained and provides all of the stimulus needed to execute. When using one of the host controller configurations (i.e. a_v8_usb_hc or a_v8_usb_hc_hi_perf), the IPS_PCB labeled UHOST_CTL will be the Host Controller and initiate all USB transactions. The USB transactions that will be performed are under user control and will be coded in the V8 firmware. The VUSB is shipped with Host Controller code to perform Chapter 9, Bus Enumeration, and ISO Audio transactions. Additional user commands for the V8 firmware can be input from the HOST_TRM.DAT file.

When using one of the target configurations (i.e. a_v8_usb or a_v8_usb_hi_perf), the behavioral Host Controller will initiate all USB transactions. The user has the ability to enable which tests they wish to perform during simulation. This allows simulation times to be kept to a minimum. The following table describes these variables and the tests they enable:

Variable Name	Programmable Control
chapter_9_tests_enabled	This test will perform a Chapter 9 test on a target device. Default value = true.
chapter_11_tests_enabled	This test will perform a Chapter 11 test on a HUB device. Default value = false.
bus_enumeration_target_enabled	This test will enumerate a single USB device or HUB. Default value = true.
bus_enumeration_hub_enabled	This test will enumerate a single USB device that is downstream from the VUSB HUB. Default value = false.
quick_check_enabled	This test runs a quick set of the USB Peripheral Silicon test. This test is designed to test things quickly and efficiently for simulation and test vector generation. Default value = true.
checklist_enabled	This test runs an exhaustive version of the USB Peripheral Silicon test. All tests are run in order and have no regards for simulation times. Default value = false.

Setting any of these variables to true will enable that specific test, while setting to false will disable that test. For more information on exactly what USB transactions take place during these tests see the HOST_CTL.V(HD) file for more information.

Two variable are used to control the amount of debugging messages the HOST_CTL.V(HD) will display. These variables are in the HOST_CTL.V(HD) file:

Variable Name	Programmable Control
verbose	This variable will increase the number of informational messages that HOST_CTL.V(HD) displays. Default value = false.
terse	This variable will cut the number of informational messages that HOST_CTL.V(HD) displays. Default value = false.

14.3 Behavioral Host Controller API

One of the powerful features of the Behavioral Host Controller is its ability to perform USB transactions via user programmable control. This programmable control includes the most basic USB packets: Start of Frame (SOF), Token, Data, and Handshake Packets. These USB packets can then be combined to execute USB Bulk, Control, Interrupt, or Isochronous Transactions. And finally the USB Transactions are combined to perform USB Device Framework. Below is each level of USB transactions are documented.

14.3.1 USB_RESET

Procedure:	usb_reset	
Parameters:	duration	Duration of the reset signal
Globals:	usb_clk	USB clock
	dplus	USB data+
	dminus	USB data-
	hc_state	USB Host Controller state
	device_address	Current USB target address
Description:	This procedure will transmit a USB reset signal (dplus and dminus Single Ended zero (SE0)) of a specified duration. The device_address global variable is set to address 0.	
Example:	usb_reset(2500 ns); -- Assert USB reset for 2500 ns	

14.3.2 USB_RESUME

Procedure:	usb_resume	
Parameters:	duration	Duration of the resume signal
Globals:	usb_clk	USB clock
	dplus	USB data+
	dminus	USB data-
	hc_state	USB Host Controller state
Description:	This procedure will transmit a USB resume signal (dplus and dminus K state followed by a low speed EOP) of a specified duration.	
Example:	usb_resume(20000000 ns); -- Assert resume signaling for 20 ms	

14.3.3 TOKEN_PACKET

Procedure:	token_packet	
Parameters:	pid	PID
	addr	USB destination address
	endpt	USB destination end point
Globals:	usb_clk	USB clock
	dplus	USB data+
	dminus	USB data-
	hc_state	USB Host Controller state
	dtoggle	Data toggle synchronization signal
Description:	This procedure will transmit a USB token packet. The token type is specified by the pid parameter. Target address, and endpoint are from the addr and endpt parameters. The data toggle variable is also initialized to DATA0. Packet Format: See USB Spec 1.0 Section 8.4.1 for more information.	
Example:	token_packet(IN_PID, device_address, endpt => 0); — In token from endpoint0	

14.3.4 BUILD_HC_PACKET

Procedure:	build_hc_packet	
Parameters:	byte_cnt	Byte count
	strt_data	Starting data value
	data_pat	Test data pattern
	seed	Random number generator seed
Globals:	hc_pckt_data	Packet data
Description:	This procedure will build a data packet in the global hc_pckt_data array. This array will hold each byte of information that will be transmitted, or will be received. This data is used by the data_in_packet and data_out_packet as compare or transmit data respectively.	
Example:	build_hc_packet(dlength, dstart, dpat, seed);	

14.3.5 DATA_IN_PACKET

Procedure:	data_in_packet	
Parameters:	exp_pid	Expected PID
	byte_cnt	Byte Count
	offset	Offset into packet data
	hand_shake	Handshake type NAK, STALL or DATA
	data_check	Enables data checking
Globals:	usb_clk	USB clock
	dplus	USB data+
	dminus	USB data-
	hc_state	USB Host Controller state
	hc_pkt_data	Expected packet data
Description:	This procedure will receive a complete data packet. The received PID is compared with the expected PID. Each byte of the data field can be compared with the expected packet data (hc_pkt_data). The data comparison is enabled when data_checking is equal to check or iso_check. Data checking can also be disabled on a byte by byte basis by setting hc_pkt_data to X_es. The offset in to the hc_pkt_data can be passed in to the data_in_packet procedure. This allows the data_in_packet to be called repeatedly by higher level routines and compute the correct index in the expected receive data. Data length checking is performed by comparing the byte_cnt parameter with the actual number of bytes received. Receive length checking is also enabled with the data_checking parameter. If called with data_check equal to iso_check it will allow 0 length ISO packets to be received error free. The CRC16 is computed and checked with 800B\h. If any of these comparisons fail an error message is displayed. Bus turn around times are also checked to be within USB specification. Packet Format: See USB Spec 1.0 Section 8.4.3 for more information.	
Example:	<pre> — Receive data in packet with data checking enabled data_in_packet(exp_pid => DATA_PID, — Expected DATA0/1 PID byte_cnt => expected_len, — Expected receive length offset => packet_offset, — Begin indexing of hc_pkt_data -- with this offset hand_shake => hand_shake, — Expected hand shake data_check => check); — Data checking is enabled </pre>	

14.3.6 DATA_OUT_PACKET

Procedure:	data_out_packet	
Parameters:	pid	PID
	byte_cnt	Byte Count
Globals:	usb_clk	USB clock
	dplus	USB data+
	dminus	USB data-
	hc_state	USB Host Controller state
	device_address	Current USB target address
Description:	This procedure will transmit a complete data packet. The pid parameter is transmitted during the PID portion of the packet. The number of bytes to be transmitted is passed in via the byte_cnt parameter. Transmit data must be built in the hc_pkt_data array before this procedure is called. Finally the 16-bit CRC is computed based on the hc_pkt_data and transmitted during the CRC portion of data packet. Packet Format: See USB Spec 1.0 Section 8.4.3 for more information.	
Example:	-- Send the tx packet data with an OUT pid and a byte count of 8 data_out_packet(DATA0_PID,16#8#);	

14.3.7 HANDSHAKE_IN_PACKET

Procedure:	handshake_in_packet	
Parameters:	exp_pid	Expected PID
Globals:	usb_clk	USB clock
	dplus	USB data+
	dminus	USB data-
	hc_state	USB Host Controller state
Description:	This procedure will receive a USB handshake packet. This procedure should be used when the behavioral host controller must receive a handshake packet. The received PID is compared with the expected PID. If this comparison fails an error message is displayed. Bus turn around times are also checked to be within USB specification. Packet Format: See USB Spec 1.0 Section 8.4.4 for more information.	
Example:	ASSERT not verbose REPORT "Check the ack on the setup packet" SEVERITY NOTE; handshake_in_packet(hand_shake);	

14.3.8 HANDSHAKE_OUT_PACKET

Procedure:	handshake_out_packet	
Parameters:	pid	PID
Globals:	usb_clk	USB clock
	dplus	USB data+
	dminus	USB data-
	hc_state	USB Host Controller state
Description:	This procedure will transmit a USB handshake packet. The pid parameter is transmitted during the PID portion of the packet. Packet Format: See USB Spec 1.0 Section 8.4.4 for more information.	
Example:	<pre>-- Acknowledge the config block ASSERT terse PORT "Acking this descriptor packet" SEVERITY NOTE; handshake_out_packet (ACK_PID);</pre>	

14.3.9 GET_DEVICE_DESC

Procedure:	get_device_desc	
Parameters:	test_num	Base test number for test progress signal
Globals:	usb_clk	USB clock
	device_address	Current USB target address
	hc_pkt_data	Transmit and expected packet data
	test_progress	Counts test bench progression
	device_max_pkt	Maximum packet size for control endpoint 0.
Description:	This procedure performs a get device descriptor control transfer, and sets the global variable device_max_pkt. Currently only certain bytes of the device descriptor data that are not likely to be changed are checked. Users can modify this data to be there expected device descriptor information for data comparison. The test_num parameter is assigned to the global variable test_progress, and incremented as the test proceeds. The get_device_desc procedure is an excellent example of how USB packet procedures can be used together to create complete USB transactions. The global device_address is used to determine the current address of the target being addressed. For more information on the device descriptor see Section 9.6.1 of the USB Spec Version 1.0.	
Example:	<pre>-- Try one setup transaction for each bRequest type from Table 9.3 of the USB -- specification. get_device_desc(16#90001#); -- Get the device descriptor</pre>	

14.3.10 GET_HUB_DESC

Procedure:	get_hub_desc	
Parameters:	test_num	Base test number for test progress signal
Globals:	usb_clk	USB clock
	device_address	Current USB target address
	hc_pckt_data	Transmit and expected packet data
	test_progress	Counts test bench progression
Description:	This procedure performs a get hub device descriptor control transfer. Users can modify this data to be there expected hub device descriptor information for data comparison. The test_num parameter is assigned to the global variable test_progress, and incremented as the test proceeds. The global device_address is used to determine the current address of the target being addressed. For more information on the device descriptor see Section 9.6.1 of the USB Spec Version 1.0.	
Example:	<pre>-- Get the hub descriptor. get_hub_desc(test_num => 16#B000#);</pre>	

14.3.11 GET_CONFIG_DESC

Procedure:	get_config_desc	
Parameters:	test_num	Base test number for test progress signal
Globals:	usb_clk	USB clock
	device_address	Current USB target address
	hc_pckt_data	Transmit and expected packet data
	test_progress	Counts test bench progression
Description:	This procedure performs a get configuration descriptor control transfer. Currently only certain bytes of the configuration descriptor data are checked. Users can modify this data to be there expected configuration descriptor information for data comparison. The test_num parameter is assigned to the global variable test_progress, and incremented as the test proceeds. The global device_address is used to determine the current address of the target being addressed. For more information on the configuration descriptor see Section 9.6.2 of the USB Spec Version 1.0.	
Example:	<pre>ASSERT false REPORT "Attempt a get configuration." SEVERITY NOTE; get_config_desc(16#9020#); -- Get the configuration descriptor</pre>	

14.3.12 GET_DEVICE_STATUS

Procedure:	get_device_status	
Parameters:	test_num	Base test number for test progress signal
Globals:	usb_clk	USB clock
	device_address	Current USB target address
	hc_pkt_data	Transmit and expected packet data
	test_progress	Counts test bench progression
Description:	This procedure performs a get device status control transfer. Currently only certain bytes of the device status data are checked. Users can modify this data to be there expected device status information for data comparison. The test_num parameter is assigned to the global variable test_progress, and incremented as the test proceeds. The global device_address is used to determine the current address of the target being addressed. For more information on the Get Status device request see Section 9.4.5 of the USB Spec Version 1.0.	
Example:	<pre>ASSERT false REPORT "Attempt a get device status." SEVERITY NOTE; get_device_status(16#9040#); -- Get the status</pre>	

14.3.13 GET_ENDPOINT_STATUS

Procedure:	get_endpoint_status	
Parameters:	test_num	Base test number for test progress signal
	endpt_num	Endpoint to determine status
Globals:	usb_clk	USB clock
	device_address	Current USB target address
	hc_pkt_data	Transmit and expected packet data
	test_progress	Counts test bench progression
Description:	This procedure performs a get endpoint status control transfer. Currently none of the endpoint status data is checked. Users can add there expected endpoint status information for data comparison. The test_num parameter is assigned to the global variable test_progress, and incremented as the test proceeds. The global device_address is used to determine the current address of the target being addressed. For more information on the Get Status device request see Section 9.4.5 of the USB Spec Version 1.0.	
Example:	<pre>ASSERT false REPORT "Do a get endpoint status to check that set feature worked" SEVERITY NOTE; get_endpoint_status(endpt_num => 1, test_num => 16#9058#); ASSERT (hc_rxpkt_data(0) = "00000001") REPORT "endpoint did not report stall status." SEVERITY FAILURE;</pre>	

14.3.14 GET_VALUE

Procedure:	get_value	
Parameters:	breq	Request type 8-configuration 10-interface
	exp_value	Expected configuration value
	wval	Configuration value to be set
	wind	wIndex used in set interface
	test_num	Base test number for test progress signal
Globals:	usb_clk	USB clock
	device_address	Current USB target address
	hc_pkt_data	Transmit and expected packet data
	test_progress	Counts test bench progression
Description:	This procedure performs a get configuration or get interface control transfer to a usb target. Currently none of the configuration or interface status data is checked. Users can add there configuration or interface status information for data comparison. The test_num parameter is assigned to the global variable test_progress, and incremented as the test proceeds. The global device_address is used to determine the current address of the target being addressed. For more information on the Get Configuration device request see Section 9.4.2 of the USB Spec Version 1.0. For more information on the Get Interface device request see Section 9.4.4 of the USB Spec Version 1.0.	
Example:	<pre>-- get configuration ASSERT false REPORT "Attempt a get configuration (expect 01)."</pre> SEVERITY NOTE; <pre>get_value(breq => 8, exp_value => 16#01#, wind => 0, test_num => 16#9080#);</pre>	

14.3.15 SET_ADDRESS

Procedure:	set_address	
Parameters:	address	Address to set in target
	test_num	Base test number for test progress signal
Globals:	usb_clk	USB clock
	hc_pckt_data	Transmit packet data
	device_address	Current USB target address
	test_progress	Counts test bench progression
Description:	This procedure performs a set address transfer to a USB target. The test_num parameter is assigned to the global variable test_progress, and incremented as the test proceeds. The global device_address is used to determine the current address of the target being addressed. When complete the device_address global variable is then set to the address parameter. For more information on the Set Address device request see Section 9.4.6 of the USB Spec Version 1.0.	
Example:	set_address(16#3f#,0); -- Set the device address to 3F.	

14.3.16 SET_VALUE

Procedure:	set_value	
Parameters:	breq	Request type 9-configuration 11-interface
	wval	Configuration value to be set
	wind	wIndex used in set interface
	test_num	Base test number for test progress signal
Globals:	usb_clk	USB clock
	device_address	Current USB target address
	hc_pckt_data	Transmit and expected packet data
	test_progress	Counts test bench progression
Description:	This procedure performs a set configuration or set interface control transfer to a USB target. The test_num parameter is assigned to the global variable test_progress, and incremented as the test proceeds. The global device_address is used to determine the current address of the target being addressed. For more information on the Set Configuration device request see Section 9.4.7 of the USB Spec Version 1.0. For more information on the Set Interface device request see Section 9.4.10 of the USB Spec Version 1.0.	

Example:	<pre>-- set_interface 1 to 1 ASSERT false REPORT "Attempt to set interface 1 to 1." SEVERITY NOTE; set_value(breq => 16#b#, wval => 1, wind => 1, test_num => 16#90a0#);</pre>
----------	--

14.3.17 SET_FEATURE

Procedure:	set_feature	
Parameters:	rtype	0-device, 1-interface, 2-endpoint.
	wval	The feature selector to be set.
	wind	The interface or endpoint number
	test_num	Base test number for test progress signal
Globals:	usb_clk	USB clock
	device_address	Current USB target address
	hc_pkt_data	Transmit and expected packet data
	test_progress	Counts test bench progression
Description:	This procedure performs a set feature control transfer to a USB target. The rtype parameter selects the destination of the set feature. This can be a device, interface, or endpoint. The wval is the feature selector to be set within the recipient device, interface, or endpoint. And wind selects the interface or endpoint number. The test_num parameter is assigned to the global variable test_progress, and incremented as the test proceeds. The global device_address is used to determine the current address of the target being addressed. For more information on the Set Feature device request see Section 9.4.9 of the USB Spec Version 1.0.	
Example:	<pre>-- set the endpoint 1 stall feature ASSERT false REPORT "Attempt a set feature stall on endpoint 1." SEVERITY NOTE; set_feature(rtype => 2, wval => 0, wind => 1, test_num => 16#9050#); -- let's go a head and check that it worked ASSERT false REPORT "Do a get endpoint status to check that the previous set feature worked" SEVERITY NOTE; get_endpoint_status(endpt_num => 1, test_num => 16#9058#); ASSERT ((hc_rxpkt_data(0) = "00000001") AND (hc_rxpkt_data(1) = "00000000")) REPORT "endpoint did not report stall status." SEVERITY FAILURE;</pre>	

14.3.18 CLEAR_FEATURE

Procedure:	clear_feature	
Parameters:	rtype	0-device, 1-interface, 2-endpoint.
	wval	The feature selector to be cleared.
	wind	The interface or endpoint number
	test_num	Base test number for test progress signal
Globals:	usb_clk	USB clock
	device_address	Current USB target address
	hc_pkt_data	Transmit and expected packet data
	test_progress	Counts test bench progression
Description:	This procedure performs a clear feature control transfer to a USB target. The rtype parameter selects the destination of the clear feature. This can be a device, interface, or endpoint. The wval is the feature selector to be cleared within the recipient device, interface, or endpoint. And wind selects the interface or endpoint number. The test_num parameter is assigned to the global variable test_progress, and incremented as the test proceeds. The global device_address is used to determine the current address of the target being addressed. For more information on the Clear Feature device request see Section 9.4.1 of the USB Spec Version 1.0.	
Example:	<pre>-- clear the endpoint 1 stall feature ASSERT false REPORT "Attempt a clear feature stall on endpoint 1." SEVERITY NOTE; clear_feature(rtype => 2, wval => 0, wind => 1, test_num => 16#9060#); ASSERT false REPORT "Do a get endpoint status to check that the previous set feature worked" SEVERITY NOTE; get_endpoint_status(endpt_num => 1, test_num => 16#9058#); -- let's go a head and check that it worked ASSERT ((hc_rxpkt_data(0) = "00000000") AND (hc_rxpkt_data(1) = "00000000")) REPORT "endpoint did not report not stalled status." SEVERITY FAILURE;</pre>	

14.3.19 GET_DATA

Procedure:	get_data	
Parameters:	rtype	USB Chapter 9/11 bmRequestType field to send
	breq	USB Chapter 9/11 bRequest field to send. Values include: 0 - get status 2 - get state 6 - get descriptor 8 - get configuration 10 - get interface
	wval	USB Chapter 9/11 wValue field to send
	wind	USB Chapter 9/11 wIndex field to send
	wlen	The length of the data packet to be returned
	check	Enable or disable data checking
	test_num	Base test number for test progress signal
Globals:	usb_clk	USB clock
	device_address	Current USB target address
	device_max_pkt	Current target max packet size
	hc_pkt_data	Transmit packet data
	hc_rxpkt_data	Get return data block
	test_progress	Counts test bench progression
Description:	This procedure is used to do variable length get requests. It was specifically designed to handle the hub class specific requests, such as get hub descriptor, get hub status and get hub port status. The rtype parameter is assigned to the bmRequestType field of the USB device request. The breq parameter is assigned to the bRequest field of the USB device request. This code is designed only to perform GET_STATUS, GET_STATE, GET_DESCRIPTOR, GET_CONFIGURATION, or GET_INTERFACE. So legal values of breq are 0, 2, 6, 8, and 10. The wval, wind, and wlen parameters are assigned to the wValue, wIndex, and wLength fields of the USB device request respectively. The wlen parameter is also used to calculate the expected size of the received packets. The data returned is checked if the check parameter is set to check. If the check parameter is set to no_check the data will not be checked. The received data is available via the hc_rxpkt_data data structure. The test_num parameter is assigned to the global variable test_progress, and incremented as the test proceeds. The global device_address is used to determine the current address of the target being addressed. The global device max packet is used to determine if the transaction must be broken into multiple IN data packets in order to receive the amount of data specified by wlen.	

Example:	<pre> -- set the expected data in the hc_pkt_data hc_pkt_data(0) := conv_std_logic_vector(16#09#,8); -- bLength hc_pkt_data(1) := conv_std_logic_vector(16#29#,8); -- bDescriptor Type hc_pkt_data(2) := conv_std_logic_vector(16#06#,8); -- Number down stream ports hc_pkt_data(3) := conv_std_logic_vector(16#0d#,8); -- wHubCharacteristics hc_pkt_data(4) := conv_std_logic_vector(16#00#,8); hc_pkt_data(5) := conv_std_logic_vector(16#02#,8); -- bPwrOn2PwrGood hc_pkt_data(6) := conv_std_logic_vector(16#00#,8); -- bContrCurrent hc_pkt_data(7) := conv_std_logic_vector(16#00#,8); -- Device Removable hc_pkt_data(8) := conv_std_logic_vector(16#3f#,8); -- PortPwrCtrlMask get_data(rtype =>16#a0#, -- bmRequestType breq => 6, -- bRequest : get descriptor wval => 16#2900#, -- wValue : Hub class wind => 0, -- wIndex wlen => 9, -- wLength : length of returned packet. check => check, -- checking enabled. test_num => test_num); </pre>
----------	---

14.3.20 SET_DIAG_DESC

Procedure:	set_diag_desc	
Parameters:	breq	The diagnostic set request type. Valid values include: 0 - set status register 2 - set endpoint register 4 - set memory
	wval	The value to write to the status or endpoint register.
	wind	The address of the memory or endpoint register.
	wlen	The length of the data packet to follow. Zero for set status and set endpoint. Equal to the byte count for set memory.
Globals:	usb_clk	USB clock
	device_address	Current USB target address
	hc_pkt_data	Transmit packet data

Description:	This procedure is used to perform a vendor specific set descriptor USB device request. When used in conjunction with Vautomation's VUSB code, Diagnostic test information can be performed during simulations. This is useful to set values in the VUSB status register, endpoint control registers, or in memory locations within the device being simulated. This information can be set by the behavioral host controller by performing a set_diag_desc. This allows the user to create powerful test procedures to set diagnostic information in the target device. The set_diag_desc is used extensively in the Peripheral Silicon USB Compliance Checklist in the behavioral host controller. The breq parameter is assigned to the bRequest field of the USB device request. The VUSB code running on the V8 is designed to decode only SET_STATUS, SET_EP_CTL, and SET_MEM requests. So legal values of breq are 0, 2, and 4. The wval parameter is assigned to the wValue field of the USB device request and contains the value to write to the VUSB status or endpoint register. The wind parameter is assigned to the wIndex field of the USB device request and contains the starting address in memory for a SET_MEM request or the address of the endpoint register for a SET_EP_CTL request. The wlen parameter is assigned to the wLength field of the USB device request and contains the number of bytes to write to memory for a SET_MEM request. For SET_STATUS or SET_EP_CTL requests it is set to 0.
Example:	<pre> ----- -- Clear all the bits in the Diagnostic register of the target device. ----- set_diag_desc(breq => 0, wval => 16#ffff#, wind => 0, wlen => 0); </pre>

14.3.21 GET_DIAG_DESC

Procedure:	get_diag_desc	
Parameters:	breq	The diagnostic get request type. Valid values include: 1 - get status register 3 - get endpoint register 5 - get memory
	wval	Should be set to zero.
	wind	The address of the memory or endpoint register.
	wlen	The length of the data packet to follow. Two for get status and get endpoint. Equal to the byte count for get memory.
Globals:	usb_clk	USB clock
	device_address	Current USB target address
	hc_pckt_data	Transmit packet data
	hc_rxpkt_data	Gets return data block.
Description:	This procedure is used to perform a vendor specific get descriptor USB device request. When used in conjunction with Vautomation's VUSB code, Diagnostic test information can be performed during simulations. This is useful to get values from the VUSB status register, endpoint control registers, or memory locations within the device being simulated. This information can be request by the behavioral host controller by performing a get_diag_desc. This allows the user to create powerful test procedures to extract diagnostic information from the target device. The get_diag_desc is used extensively in the Peripheral Silicon USB Compliance Checklist in the behavioral host controller. The breq parameter is assigned to the bRequest field of the USB device request. The VUSB code running on the V8 is designed to decode only GET_STATUS, GET_EP_CTL, and GET_MEM requests. So legal values of breq are 1, 3, and 5. The wval parameter is assigned to the wValue field of the USB device request and should always be set to zero. The wind parameter is assigned to the wIndex field of the USB device request and contains the starting address in memory for a GET_MEM request or the address of the endpoint register for a GET_EP_CTL request. The wlen parameter is assigned to the wLength field of the USB device request and contains the number of bytes to read from memory for a GET_MEM request. For GET_STATUS or GET_EP_CTL requests it is set to 2. The data returned is not checked, but is available via the hc_rxpkt_data data structure.	
Example:	<pre>-- Read the target status to confirm suspend and resume get_diag_desc(breq => 1, wval => 0, wind => 0, wlen => 2); -- Check the diagnostic register return data to insure that the target had gone -- into suspend state. status_rg := hc_rxpkt_data(0); ASSERT ((status_rg and "11011111")= "00000000") REPORT "errors detected by target." SEVERITY FAILURE; ASSERT ((hc_rxpkt_data(1) AND "00000001") = "00000001") REPORT "Target did not go into suspend state." SEVERITY FAILURE;</pre>	

14.4 Executing the Test Bench in VHDL

Running a simulation of the VUSB Test Bench is obviously simulator and language dependent. Some modifications might be needed to the supplied makefile for proper execution with your simulation environment, see the makefile file for more information. To run the VHDL test bench, first compile the design for the target simulator, then load the proper configuration into the simulator and run. Below is a brief description of how to run the test bench on the Model Technology VHDL simulator.

1) Use the Makefile to compile the VUSB for simulation. Make sure the environment variables are correctly set before you run. See the comments in the Makefile for more information. To compile for MTI simulation type:

```
make sim or make mti
```

This will create a working directory if one does not exist, then compile all the VHDL based upon the file dependencies. The supplied V8 software is also compiled and copied to RAM128k8.HEX. The V8 BISTROM code is also compiled and copied to BISTROM.HEX.

2) Next you should start the MTI simulator. Change to your working directory and then load the desired configuration you wish to simulate. For the VUSB that will be one of the following configurations: a_v8_usb, a_v8_usb_hi_perf, a_v8_usb_hc, or a_v8_usb_hc_hi_perf. See Section 13.3 for more information on the various VUSB configurations.

3) Once the desired configuration has been loaded execute on the supplied MTI command files. If you are simulating a target configuration (i.e. a_v8_usb or a_v8_usb_hi_perf) execute startup.do. If you are simulating a host controller configuration (i.e. a_v8_usb_hc or a_v8_usb_hc_hi_perf) execute start_hc.do. These command files will display a number of signals for observation during simulation and then start the simulation.

5) If the simulation has completed successfully the VUSB test bench will print:

```
"Note: Successful completion of USB test bench."
```

This indicates that the VUSB has successfully completed all enabled tests. If the simulation was unsuccessful the VUSB test bench will print:

```
"Failure: Error message describing failure."
```

This indicates that a specific USB test has failed. The point that the code failure occurred will be displayed in the MTI source window. The user can determine the source of the failure and during what test it occurred. Generally test bench failures can be attributed to incorrect expected values for USB descriptors or other various bugs in a user's V8 firmware. The user can then use this information to debug the problem, for more information on VUSB debugging see Section 15.1, in Tips and Techniques. If the problem persists consult your VAutomation synthesis support consultant.

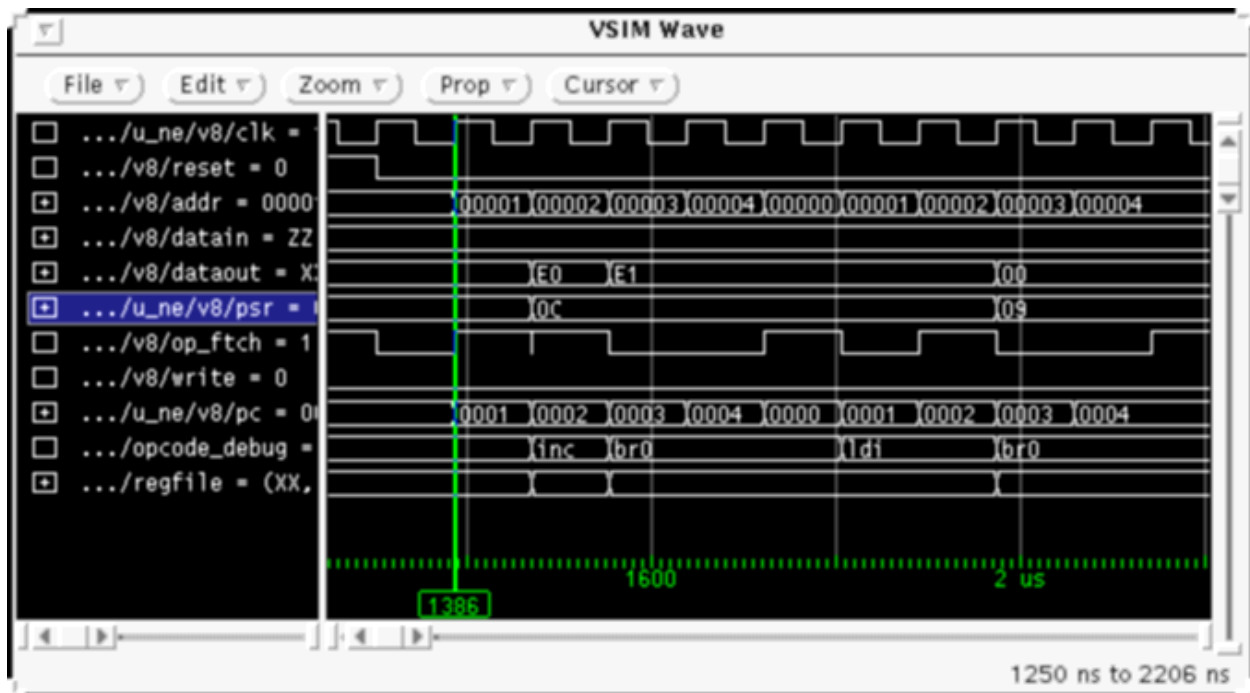


Figure 14 Sample Simulator Waveform Output

14.5 Executing the Test Bench in Verilog

To run the Verilog test bench is very similar to VHDL, with the exception that the correct architecture files need to be used. To do this the makefile will copy the correct filename_arch.v files to filename.v then compile the design for the target simulator, then run the simulator. For example to run a Verilog simulation that would use the correct architectures needed to implement a_v8_usb. The makefile will copy testbnch_target.v to testbnch.v, vusb_top_async.v to vusb_top.v, and v8_lib/c1k_ctl_fpga.v to v8_lib/c1k_ctl.v. This will correctly select the correct architectures needed to implement the a_v8_usb configuration. The copied files are touched to update there timestamps, then the make sim command is executed. This will compile the Verilog files for simulation. The file CUR_ARCH.TXT will always contain the current VUSB configuration that is being used. The makefile will echo this to your display during Verilog builds. Below is a brief description of how to run the test bench on the Model Technology Verilog simulator.

1) Use the Makefile to compile the VUSB for simulation. Make sure the environment variables are correctly set before you run. See the comments in the Makefile for more information. To compile the for MTI simulation type:

```
make a_usb_hsdev_async    - high speed device with asynchronous V8 interface
make a_usb_hsdev_sync     - high speed device with synchronous V8 interface
make a_usb_hc_hsdev_async - host controller with high speed device and asynchronous V8
                           interface
make a_usb_hc_hsdev_sync  - host controller with high speed device and synchronous V8
                           interface.
make a_usb_hc_system_sync - host controller with hub, low speed device and synchronous
```

These commands will create a working directory if one does not exist, then compile all the VHDL based upon the file dependencies. The supplied V8 software is also compiled and copied to RAM128k8.HEX. The V8 BISTROM code is also compiled and copied to BISTROM.HEX. The

HEX files are then converted to MEM files for Verilog simulation.

2) Next you should start the MTI simulator. Change to your working directory and then load the testbnch module. Remember the current configuration will be loaded based on the files copied from the previous step and displayed in CUR_ARCH.TXT See Section 13.3 for more information on the various VUSB configurations.

3) Once the desired configuration has been loaded execute on the supplied MTI command files. If you are simulating a target configuration (i.e. a_v8_usb or a_v8_usb_hi_perf) execute startup.do. If you are simulating a host controller configuration (i.e. a_v8_usb_hc or a_v8_usb_hc_hi_perf) execute start_hc.do. These command files will display a number of signals for observation during simulation and then start the simulation.

5) If the simulation has completed successfully the VUSB test bench will print:

"Note: Successful completion of USB test bench."

This indicates that the VUSB has successfully completed all enabled tests. If the simulation was unsuccessful the VUSB test bench will print:

"Failure: Error message describing failure."

This indicates that a specific USB test has failed. The point that the code failure occurred will be displayed in the MTI source window. The user can determine the source of the failure and during what test it occurred. Generally test bench failures can be attributed to incorrect expected values for USB descriptors or other various bugs in a users V8 firmware. The user can then use this information to debug the problem, for more information on VUSB debugging see Section 15.1, in Tips and Techniques. If the problem persist consult your VAutomation synthesis support consultant.

Several Cadence Verilog-XL run files are included with the VUSB. These files will directly read in the correct file architectures to implement any of the VUSB configurations. These files are documented below:

File Name	Equivalent VUSB Configuration
a_usb_hsdev_async.run	Asynch configuration for USB targets.
a_usb_hc_hsdev_async.run	Asynch configuration for USB host controllers.
a_usb_hsdev_sync.run	High Performance configuration for USB targets.
a_usb_hc_hsdev_sync.run	High Performance configuration for host controllers.
a_usb_hc_system_sync.run	High Performance configuration for host controller with hub and low speed device.

14.6 V8 C-Assembly language software

The VUSB core comes with a sample application of a USB audio device. The assembly source code is provided in the SOFTWARE subdirectory. Use the source code as an example of how to code your application. Brief descriptions of the files are listed below. See the headers of the files for more detail.

The BISTROM.ASM and USB_C.ASM have been compiled and copied to BISTROM.HEX and

RAM128k8.HEX respectively. These hex files are loaded into behavioral RAM model for program execution during simulation. For Verilog users the HEX files are converted to MEM files. The BISTROM_HW.ASM and USB_CHW.ASM are used when you compile with the WANT_HARDWARE conditional compile switch true. The WANT_HARDWARE switch should only be used when the compiler output will be used on actual hardware. Enabling the WANT_HARDWARE switch will enable execution of sections of code that would take to long in simulation, for example the CRC check of downloaded code.

File Name	Description
BIST	128 byte Built-in-Self test.
BISTROM	Combines the BIST and BOOT programs.
BISTROM_HW	Exactly the same code as BISTROM except it was compiled with the WANT_HARDWARE compile switch set.
BOOT	384 byte Boot code. Reads data from serial I2C EEPROMs and copies it into SRAM. If the checksum of the EEPROMs fails, then it will wait and download new code from the UART.
I2C	I2C serial EEPROM bus drivers.
USB_C	VUSB assembly source code of demo application.
USB_CHW	Exactly the same code as USB_C except it was compiled with the WANT_HARDWARE compile switch set.

The file USB_C.ASM was created from C code using VAutomation's C Compiler. The compiled C code was then disassembled and is shipped along with the V8 Assembler for VUSB customers. The original C code is available to customers who purchase the C Compiler.

15 Tips and Techniques

15.1 Debugging VUSB software in an HDL simulator

Simulations of the VUSB in an HDL simulator can provide significant time-to-market advantages by enabling hardware-software co-design, debug and integration. The first step in this process is to build a detailed system-level simulation environment. The VUSB ships with a fairly complete environment for the IntelliCore™ Prototype System which can be easily modified to match your own environment.

Once the simulation environment is ready, software can be loaded via an Intel Hex file into the RAMs or ROMs in the system and debug can begin in the HDL simulator. This approach has been used quite extensively by VAutomation in debugging the drivers for the VUSB Core. One advantage of the HDL simulator is that you have full access to all nodes in the design at all times. This allows quick and easy debug of state machines or other hardware designs. However, when debugging software, the large amount of data presented by the simulator can make following program flow much like finding a needle in a haystack. One trick we have found to quickly locate software routines which are currently being debugged is to add a STP (4-7) opcode to the start of a routine and a CLP opcode to the end. This technique makes it very easy to find the routine being tested in the simulator waveform viewer by simply

displaying the PSR register bits individually. You can then quickly zoom in on regions which are suspect and complete the debug process by viewing the address and data busses as well as the internal registers. Remember that the upper 4 bits of the PSR are basically under software control and you can think of these bits as single bit flags. Note that viewing bit 3 of the PSR makes it very obvious when the V8 is processing interrupts and can be used to calculate how much overhead is available in the various interrupt service routines.

Another very useful debugging technique is to create a list file of the V8 program counter, opcode, program status register, and the V8 opcode_debug signals. The list file can then be compared with the listing file created from the V8ASM or CV8 programs. To check program execution. Below is an example of how to create a listing in the MTI simulator:

```
list -width 4 -hex -notrig /ips1/uxil1/v8/cpu/pc="pc"  
list -width 4 -hex -notrig /ips1/uxil1/v8/cpu/opcde="opc"  
list -width 4 -hex -notrig /ips1/uxil1/v8/cpu/psr="psr"  
list -width 8 /ips1/uxil1/v8/cpu/o/opcode_debug="opcode"
```

15.2 Interfacing to Off-Chip SRAM

When interfacing the VUSB to off-chip SRAM, you may have difficulties meeting the address and data hold times relative to the write enable. To solve this problem, gate WRITE with the clock to create a clean WE_N which meets hold time of the SRAM. Gating WRITE with the clock is safe because WRITE comes directly from a flop. The clock will force WR_N to the SRAM high during the first half of the clock cycle and drives WR_N high with plenty of hold time available for address and data. See the ASIC_IOS.V(HD) module for an example of this as implemented in the IntelliCore™ Prototype System.

```
WR_N <= NOT (WRITE AND NOT CLK);
```

16 IntelliCore™ Prototype System

The VUSB can be evaluated using VAutomations IntelliCore™ Prototype System (IPS). Details on the IPS can be found on our web site at [HTTP://www.vautomation.com](http://www.vautomation.com).